Proactive Multimodal Academic Support System

A report submitted in partial fulfillment of the requirements
for

the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Vemireddy Usha (2211CS010595)

Under the guidance
of

Mr. Hari Battana
Assistant Professor

Department of Computer Science and Engineering
School of Engineering

MALLA REDDY UNIVERSITY
Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2026

Proactive Multimodal Academic Support System

A project report submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

In

Computer Science and Engineering

By

Vemireddy Usha (2211CS010595)

Under the guidance of

Mr. Hari Battana

Assistant Professor

Department of Computer Science and Engineering

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2026

MALLA REDDY UNIVERSITY
(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project report **entitled "Proactive Multimodal Academic Support System"**, submitted by **Vemireddy Usha (2211CS010595)** towards the partial fulfillment for the award of Bachelor's Degree in Computer Science and Engineering. Malla Reddy University, Hyderabad, is a record of Bonafide work done by him/ her. The results embodied in the work are not submitted to any other University or Institute for award of any degree or diploma.

Internal Guide
Mr. Hari Battana
Assistant Professor

Dean / Head of the Department
Dr. Meeravali Shaik
DEAN-CSE

External Examiner

DECLARATION

I hereby declare that the project report entitled "**Proactive Multimodal Academic Support System**" has been carried out by me and this work has been submitted to the Department of Computer Science and Engineering, Malla Reddy University, Hyderabad in partial fulfillment of the requirements for the award of degree of Bachelor of Technology. I further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place: Hyderabad

Date:

Vemireddy Usha 2211CS010595

ACKNOWLEDGEMENT

I extend my sincere gratitude to all those who have contributed to the completion of this project report. Firstly, I would like to extend my gratitude to **Dr. V. S. K Reddy**, Vice-Chancellor, for his visionary leadership and unwavering commitment to academic excellence.

I would also like to express my deepest appreciation to my project guide **Mr. Hari Battana**, Assistant Professor, whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout the course of this project for successful outcomes.

I would like to sincerely thank my Project Coordinator, **Mr. Hari Battana**, for his constant support, coordination, and encouragement throughout the project.

I am also grateful to **Dr. Shaik Meeravali**, Dean of the Department of Computer Science and Engineering, for providing me with the necessary resources and facilities to carry out this project.

I am deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

Vemireddy Usha (2211CS010595)

ABSTRACT

The Proactive Multimodal Academic Support System (PMASS) is an intelligent, cloud-native university companion designed to enhance academic productivity, personalized learning, and institutional support through context-aware AI. The system integrates multimodal interaction capabilities—text, images, schedules, and reminders—with real-time user data such as department timetables and academic tasks to deliver precise, role-specific assistance for students, faculty, and administrators. By employing context-fusion techniques, role-based access control, and secure service-level operations, PMASS transforms a general large language model into a specialized campus assistant capable of academic advising, lecture preparation, study planning, and administrative automation. Built within free-tier cloud constraints, the platform introduces innovative reliability mechanisms, including multi-key load balancing and a heartbeat CRON strategy to maintain continuous backend responsiveness. Additionally, the system supports AI-assisted lecture generation, progress tracking, and institutional knowledge integration, thereby bridging gaps between generic AI tools and domain-specific educational needs. PMASS demonstrates how proactive, multimodal, and secure AI systems can improve decision-making, engagement, and efficiency across higher education environments while remaining scalable and cost-effective. Furthermore, the architecture is designed for extensibility and future innovation, enabling integration with advanced analytics, collaborative knowledge bases, and enhanced multimodal processing such as diagram understanding and handwriting recognition. By supporting automated moderation, smart scheduling, and adaptive recommendations, the system fosters a safer, more organized, and data-driven campus ecosystem. This work highlights the potential of human-centered AI to act not only as a reactive assistant but as a proactive academic partner that anticipates user needs and continuously evolves with institutional requirements.

CONTENTS

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

The rapid digitization of higher education has transformed how universities operate, communicate, and deliver academic services. Technologies such as learning management systems, digital portals, AI chatbots, and mobile applications are now essential. However, many institutions still rely on fragmented systems where scheduling, academic resources, and administrative services function independently. This fragmentation increases cognitive load, creates information silos, and reduces overall efficiency.

Advancements in Large Language Models (LLMs) and multimodal AI offer new opportunities for intelligent academic support. AI-driven advising systems can assist with curriculum planning and decision-making, while multimodal models enable interaction across text, images, voice, and structured data. Despite their potential, implementing these systems in real-world universities presents challenges.

A key limitation is the lack of institutional grounding. Without access to specific data such as schedules, policies, and academic resources, AI systems often generate generic responses. Integrating structured data with semantic retrieval can improve accuracy and relevance. Additionally, infrastructure issues like API rate limits and serverless latency can affect system reliability.

Security and governance are also critical concerns. Academic systems must protect sensitive data through strict access control while ensuring transparency and preserving faculty roles. AI should enhance, not replace, human stakeholders.

To address these challenges, the Proactive Multimodal Academic Support System (PMASS) is proposed as a unified, mobile-first framework. It combines a scalable backend with multimodal AI to deliver proactive, context-aware assistance. By integrating real-time institutional data with intelligent processing, PMASS provides personalized, reliable, and secure academic support, improving overall campus efficiency.

1.1 PROBLEM DEFINITION

The rapid digital transformation of higher education has led to the adoption of various technology-driven systems, including learning platforms, scheduling tools, advisory portals, campus apps, and AI assistants. However, these systems often operate as isolated modules rather than a unified ecosystem. This fragmentation disrupts seamless information flow across academic and administrative services, forcing users to rely on multiple disconnected platforms. As a result, students and faculty face increased cognitive burden, reduced productivity, and limited access to context-aware guidance.

Despite advancements in Large Language Models (LLMs), multimodal AI, and retrieval-augmented generation, existing university implementations remain largely generic and insufficiently grounded in institutional data. Most AI-based academic tools function as standalone chat interfaces without real-time access to personalized schedules, policies, or academic requirements. Consequently, they generate generalized responses instead of actionable, institution-specific guidance.

Infrastructure challenges further impact deployment. Many institutions depend on low-cost cloud services that are prone to API rate limits, serverless cold starts, and service interruptions. These limitations reduce reliability and responsiveness, affecting user trust. Additionally, reliance on single AI service endpoints increases the risk of system-wide disruptions during high usage.

Security and governance concerns also play a crucial role. Academic systems handle sensitive data, requiring strict authentication and role-based access control. Existing AI assistants often lack fine-grained permissions, increasing the risk of unauthorized access. Moreover, AI systems must support—not replace—faculty and administrators, ensuring transparency and accountability.

Therefore, there is a need for a unified, context-aware, reliable, and secure academic support system that integrates fragmented campus services into a single platform. Such a system should combine real-time data integration, personalized

context, resilient infrastructure, and secure governance. This need motivates the development of the Proactive Multimodal Academic Support System (PMASS), designed to provide a cohesive, scalable, and institutionally grounded solution for intelligent campus digitization.

1.2 OBJECTIVE OF PROJECT

The primary objective of the Proactive Multimodal Academic Support System (PMASS) is to develop a unified, intelligent, and mobile-first campus platform that overcomes the limitations of fragmented university systems. It aims to improve academic support, efficiency, and user experience for students, faculty, and administrators. The key objectives are:

1. **Context-Aware Academic Assistance**

   Integrate institutional data (schedules, calendars, deadlines, user data) to deliver personalized guidance using multimodal AI (text, voice, image).

2. **Proactive Data Orchestration**

   Implement context-fusion to aggregate and process relevant data before AI responses, ensuring real-time accuracy.

3. **Resilient AI Infrastructure**

   Enhance reliability through API load balancing, failover mechanisms, and latency reduction strategies.

4. **Secure Role-Based Access and Governance**

   Apply authentication, role-based authorization, and data access control to protect sensitive information and ensure compliance.

5. **Enhanced Stakeholder Support**

   Support students (chat, planning, tours), faculty (AI lecture tools), and administrators (timetable and role management).

6. **Multimodal AI Integration**

   Combine text, image, and voice processing with vectorized data for accurate and interactive assistance.

7. **Scalable Mobile-First Design**

   Use a Flutter frontend with a Node.js/Supabase backend for scalable, high-performance, cross-platform access.

1.3 LIMITATIONS OF PROJECT

1. Dependence on accurate and continuously updated institutional data; if schedules, policies, or academic records are outdated or incomplete, the system may generate incorrect or misleading guidance, reducing its reliability and user trust.
2. Integration complexity with existing university systems and legacy platforms, which may have different data formats, APIs, or limited interoperability, making seamless data exchange and system unification difficult to achieve.
3. Performance limitations caused by API rate limits, serverless cold-start latency, and cloud service interruptions, which can affect system responsiveness and degrade the real-time user experience.
4. Reliance on third-party AI services and cloud providers, leading to potential issues such as service unavailability, vendor dependency, data handling concerns, and increased operational costs over time.
5. Limited accuracy in handling ambiguous, domain-specific, or highly specialized academic queries, as AI models may lack deep contextual understanding or produce generalized responses in complex scenarios.
6. Data privacy and security risks associated with handling sensitive student, faculty, and institutional data, requiring strong protection mechanisms to prevent breaches or misuse.
7. Need for strict role-based access control and governance policies to ensure that users can only access authorized data, which increases system design complexity and maintenance effort.
8. High initial development, deployment, and continuous maintenance costs, including infrastructure setup, data integration pipelines, and regular system updates.
9. User adoption challenges due to resistance to new technologies, lack of awareness, or insufficient training among students, faculty, and administrators.
10. Dependence on stable internet connectivity for accessing real-time services, which

may limit usability in low-network or offline environments.

11. Scalability challenges during peak usage periods, where handling a large number of concurrent users may require additional infrastructure, increasing cost and complexity.
12. Potential reduction in human interaction and over-reliance on AI systems, which may impact decision-making quality and reduce the role of faculty guidance if not properly balanced.

CHAPTER 2
FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

2.1 TECHNICAL FEASIBILITY

The PMASS system is technically feasible as it is built using well-established and widely supported technologies such as Flutter for cross-platform mobile application development, Node.js for scalable backend services, and Supabase for real-time database management and authentication. The integration of Large Language Models (LLMs) and multimodal AI (text, voice, and image processing) can be achieved through existing APIs and frameworks. Additionally, the use of semantic retrieval techniques and vector databases enables context-aware and personalized responses. Modern cloud infrastructure supports deployment, scalability, and continuous integration. However, certain challenges such as API rate limits, serverless cold-start latency, and integration with existing legacy systems must be carefully addressed through optimization techniques like caching, load balancing, and failover mechanisms.

2.2 ECONOMICAL FEASIBILITY

The system is economically feasible as it leverages cost-effective and partially free cloud services, along with open-source tools, which significantly reduce the initial development cost. The use of scalable cloud infrastructure ensures that resources are utilized efficiently based on demand, avoiding unnecessary expenditure. Although there may be additional costs during scaling—such as increased API usage, storage, and server resources—the long-term benefits outweigh these expenses. PMASS reduces administrative overhead, minimizes manual processes, and improves overall operational efficiency, leading to cost savings for the institution. Furthermore, the modular architecture allows gradual investment and phased implementation, making it financially manageable for educational institutions.

2.3 SOCIAL FEASIBILITY

PMASS is socially feasible as it is designed to enhance the overall academic experience for students, faculty, and administrators. By providing a unified and intelligent platform, it reduces the need to navigate multiple systems, thereby lowering cognitive load and improving user convenience. The system promotes better access to academic information, personalized guidance, and efficient communication across departments. It also supports inclusivity through multimodal interaction, enabling users to interact via text, voice, or images based on their preference. With proper training and awareness, users are likely to adopt the system easily. Moreover, the system is designed to augment human roles rather than replace them, ensuring that faculty authority and human interaction are preserved while improving productivity and collaboration within the academic environment.

CHAPTER 3

LITERATURE SURVEY

3.1 RELATED WORK

AI-Based Academic Assistance Systems | Research Studies and Applications

The use of Artificial Intelligence in higher education has gained significant attention in recent years. AI-powered academic assistants and advising tools have been developed to support students in curriculum planning, doubt clarification, and academic decision-making. Systems such as Advisely demonstrate how Large Language Models (LLMs) can provide conversational guidance to students. These systems improve accessibility and automate routine academic queries. However, most of these tools rely on static or generic datasets and are not deeply integrated with institutional databases. As a result, they often fail to provide personalized and context-aware recommendations aligned with real-time academic data.

Multimodal AI Systems in Education | Intelligent Learning Support

Recent advancements in multimodal AI have enabled systems to process text, voice, and image inputs, making academic assistance more interactive and user-friendly. Models such as Google Gemini support reasoning across multiple data formats and enhance user engagement. These systems are useful in applications like virtual tutoring, lecture assistance, and visual problem-solving. However, current implementations are primarily experimental or standalone applications and lack integration with institutional workflows such as timetables, course structures, and administrative processes.

Retrieval-Augmented Generation and Semantic Search | Context-Aware AI

Retrieval-Augmented Generation (RAG) has been widely adopted to improve the factual accuracy of AI systems by combining language models with external knowledge sources. Technologies like pgvector enable semantic search over large datasets, improving information retrieval. While these approaches enhance response quality, most systems retrieve data from static document repositories rather than

dynamic institutional databases. They also do not effectively combine user-specific data (such as schedules or deadlines) with semantic retrieval, limiting their ability to generate truly personalized academic guidance.

Cloud-Based Deployment of AI Systems | Infrastructure Challenges

The deployment of AI-based systems in educational environments often depends on cloud infrastructure. Many institutions rely on cost-effective or free-tier services, which introduce challenges such as API rate limits, serverless cold-start delays, and service interruptions. These limitations affect system reliability and responsiveness, particularly during peak usage. Additionally, centralized dependence on single AI service providers increases the risk of system-wide failures, highlighting the need for resilient and distributed infrastructure strategies.

Security and Governance in Academic Systems | Data Protection Concerns

Handling sensitive academic data requires robust security and governance mechanisms. Existing AI-based campus systems often lack fine-grained access control and consistent enforcement of role-based permissions. Techniques such as Row-Level Security (RLS) are not fully implemented, leading to potential risks of unauthorized data access. Furthermore, many systems do not integrate governance policies across all layers, including frontend, backend, and AI processing, reducing transparency and accountability.

Need for Integrated Intelligent Campus Systems

From the existing research, it is evident that while AI, multimodal systems, and semantic retrieval technologies have advanced significantly, their application in higher education remains fragmented and limited. There is a clear need for a unified system that integrates institutional data, multimodal AI, resilient infrastructure, and secure governance into a single platform. The proposed Proactive Multimodal Academic Support System (PMASS) addresses these gaps by providing context-aware, personalized, and institutionally grounded academic support within a cohesive and scalable framework.

3.2 EXISTING SYSTEM

Fragmented University Digital Ecosystem: Current university digital infrastructures are highly fragmented, where scheduling systems, learning management platforms, advisory portals, and campus navigation tools operate independently rather than as part of a unified ecosystem. This lack of integration disrupts seamless information flow across academic and administrative services. As a result, students and faculty are required to navigate multiple platforms to access information, leading to increased cognitive load and inefficiencies. Moreover, such systems provide limited context awareness, as data is scattered across different sources, resulting in delays and incomplete understanding. The absence of system interoperability also affects operational continuity, creating inefficiencies in workflows. Additionally, existing platforms offer constrained personalization, as they deliver generic guidance without adapting dynamically to individual user needs or role-specific contexts.

Disadvantages of existing system:

1. Increased cognitive load due to multiple platforms
2. Inefficient academic and administrative workflows
3. Lack of real-time data integration
4. Limited personalization of services

CHAPTER 4

METHODOLOGY

4.1 PROPOSED SYSTEM

The Proactive Multimodal Academic Support System (PMASS) is designed to address the limitations of existing fragmented university digital ecosystems. PMASS integrates institutional data, multimodal AI capabilities, resilient infrastructure, and secure governance into a unified, mobile-first platform to provide context-aware, personalized academic support for students, faculty, and administrators.

1. **System Overview**

   PMASS is a mobile-first framework built using Flutter for the frontend and Node.js/Supabase for the backend. It uses multimodal AI models such as Google Gemini to process text, voice, and image inputs. The system follows a proactive approach by gathering contextual data like timetables, deadlines, and institutional documents before generating responses, ensuring accurate and personalized guidance.

Feature	Google Classroom	Moodle (LMS)	University Website	Proposed System
Primary Focus	Assignment/Content	Course Management	Static Information	**Holistic Campus Life**
Real-time Updates	Moderate (Email)	Low	Low	**High (Push/Sockets)**
AI Assistant	No	No (Plugin required)	No	**Yes (Gemini RAG)**
Campus Nav	No	No	2D Maps	**3D Virtual Tour**
Mobile UX	Good	Generic/Web-view	Poor	**Native (Flutter)**
Personalization	Class-based	Course-based	None	**Role & Context-based**

2. **Key functional modules**

   The system includes student, faculty, and administrative modules. Students receive AI-based chat support, study planning, and navigation assistance. Faculty are supported with

lecture preparation and student insights. Administrators can manage timetables, roles, and system monitoring through centralized tools.

3. **Resilient infrastructure**

   PMASS ensures system reliability using multi-key API load balancing, failover mechanisms, and keep-alive techniques to reduce latency and prevent service interruptions. The scalable backend supports multiple users and enables fast, context-aware responses.

4. **Security and governance**

   The system implements strong security measures such as role-based access control (RBAC), row-level security (RLS), and JWT-based authentication. These ensure secure data handling, controlled access, and compliance with institutional policies.

4.2 MODULES

1. **Flutter Module** – Handles the frontend mobile application, providing a user-friendly interface for students, faculty, and administrators. It supports cross-platform access and enables interaction through text, voice, and visual components.
2. **Node.js Module** – Acts as the backend server, managing API requests, processing business logic, and coordinating communication between the frontend, database, and AI services.
3. **Supabase Module** – Manages database operations, including storage of institutional data such as timetables, deadlines, and user information. It also provides built-in authentication and real-time data synchronization.
4. **AI Module (Google Gemini)** – Processes multimodal inputs like text, voice, and images, and generates intelligent, context-aware responses for academic assistance using advanced language models.
5. **Vector Database Module** – Stores embeddings of institutional documents and enables semantic search, allowing the system to retrieve relevant information and improve the accuracy of AI responses.

6. **Authentication Module** – Ensures secure access to the system using login mechanisms, JWT tokens, and role-based access control (RBAC), protecting sensitive academic and user data.
7. **API Integration Module** – Connects all system components by enabling communication between frontend, backend, AI services, and databases through well-defined APIs.
8. **Cloud Infrastructure Module** – Supports deployment and scalability of the system, handling load balancing, failover mechanisms, and ensuring high availability and performance.

4.3 ARCHITECTURE

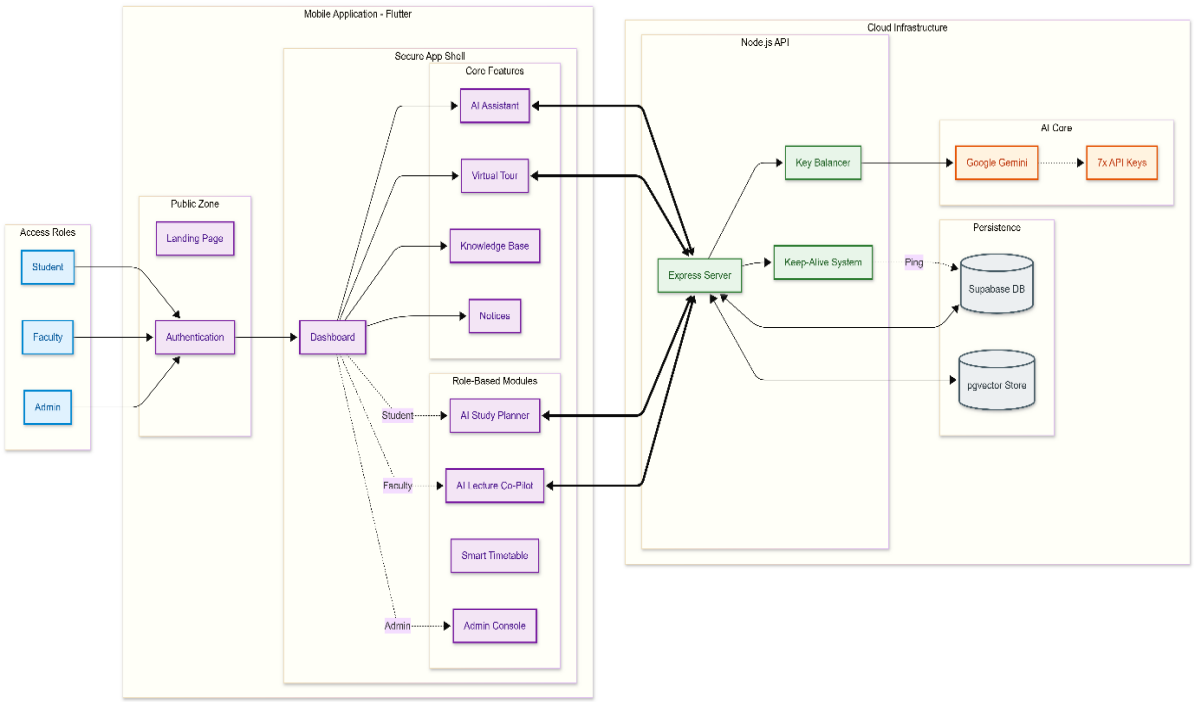


Fig.4.3.1: This architecture presents PMASS as a role-based, mobile-first academic platform where Students, Faculty, and Admins access a secure Flutter application backed by a Node.js API layer. Core and role-specific features route requests through an Express server that enforces authentication, load-balances across multiple Google Gemini API keys, and maintains reliability via a Keep-Alive system. Institutional data and knowledge are grounded using Supabase (PostgreSQL) and pgvector-based semantic retrieval, enabling context-aware, resilient AI assistance across the university ecosystem.

4.4 METHODS & TECHNIQUES

4.4.1 Context-Aware Data Orchestration

This method focuses on collecting and combining user-specific and institutional data such as timetables, academic deadlines, departmental details, and user roles before invoking the AI model. By aggregating this contextual information in advance, the system ensures that responses are accurate, personalized, and relevant to real-time academic scenarios. This approach overcomes the limitations of traditional systems that rely on generic or incomplete data

4.4.2 Retrieval-Augmented Generation (RAG)

RAG enhances the performance of AI by integrating external knowledge sources with language models. Instead of generating responses purely based on pre-trained knowledge, the system retrieves relevant information from institutional documents, databases, and vector storage, and then uses it to generate context-aware answers. This improves factual accuracy, reduces hallucination, and ensures institution-specific guidance.

4.4.3 Multimodal Interaction Technology

The system supports multiple input formats such as text, voice, and images, enabling flexible and user-friendly interaction. This is achieved using advanced AI models like Google Gemini, which can process and understand different data types. Multimodal capability enhances accessibility and allows users to interact with the system in a more natural and efficient manner.

4.4.4 Role-Based Access Control and Security

This method ensures that system access is restricted based on user roles such as student, faculty, and administrator. It uses authentication mechanisms like JWT tokens along with role-based permissions and row-level security policies to protect sensitive data. This approach maintains data privacy, prevents unauthorized access, and ensures compliance with institutional governance requirements.

4.4.5 Scalable Cloud and Backend Technologies

The system is built using modern technologies such as Flutter for frontend development, Node.js for backend services, and Supabase for database and authentication management. Cloud deployment platforms support scalability, load balancing, and failover mechanisms to ensure high availability and performance.

CHAPTER 5

DESIGN

5.1 UML DIAGRAM

Sequence diagram:

This sequence diagram illustrates an AI campus assistant workflow. A user asks a question in the mobile app, which sends a chat request to the backend API. The server creates embeddings via Gemini, retrieves relevant knowledge base articles and the user's timetable from Supabase, and constructs a system prompt. Gemini then generates a response, returned as JSON to the app, which displays the answer: the user's next class and time.

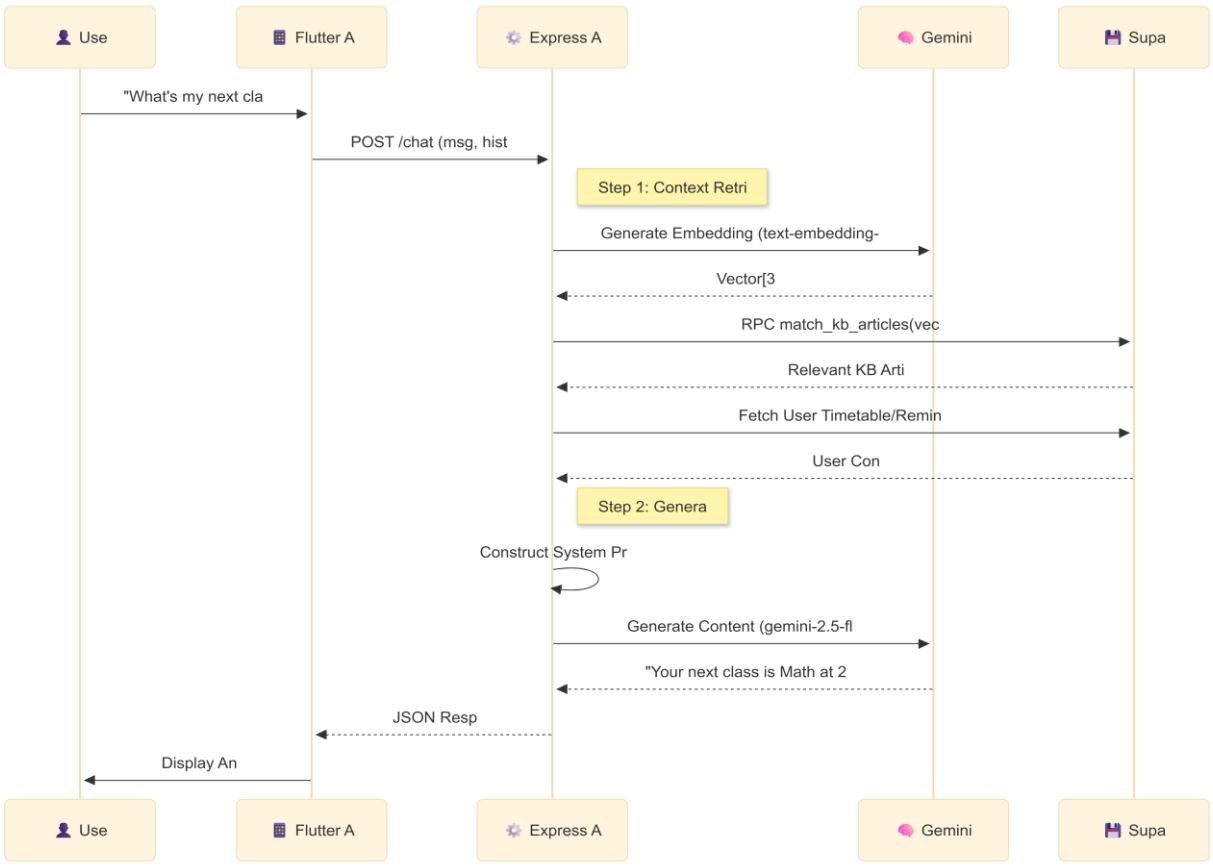


Fig.5.1.1 Sequence Diagram

Class diagram:

The diagram represents a class model for a campus assistant system. A User has login and logout functions and is associated one-to-one with a Profile containing personal and academic details. A User can send multiple ChatMessages, which store message content and timestamps. The Profile allows viewing or editing multiple Timetable entries, which store class schedule details and support updating time slots.

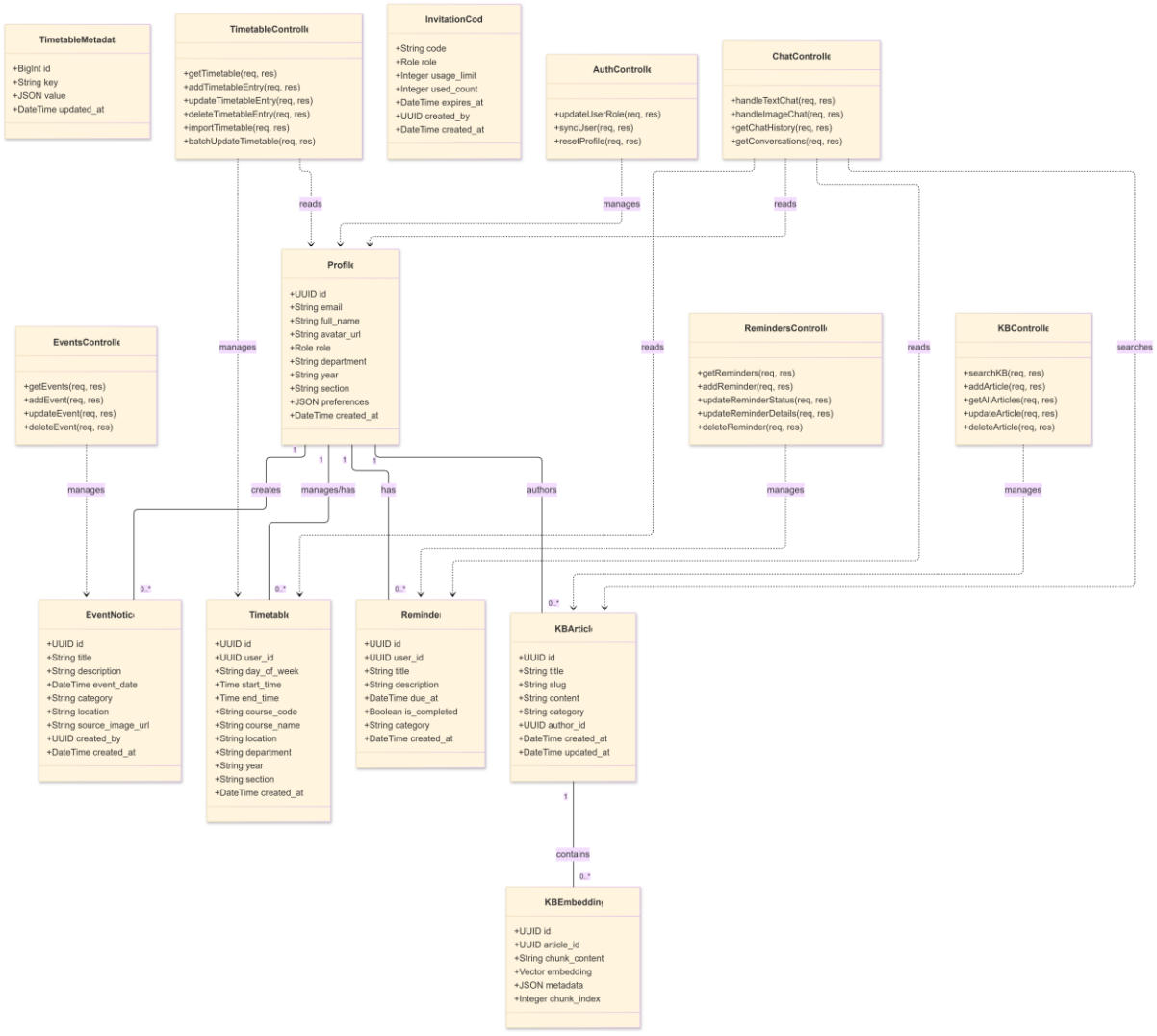


Fig.5.1.2 Class diagram

Activity diagram:

This flowchart illustrates the AI chat processing pipeline of a campus assistant. After a user

sends a message, the system detects whether it is text or image. Images are processed using Gemini Vision, while text undergoes intent analysis (next class, rules, or general queries). Relevant timetable or knowledge base data is retrieved via embeddings and vector search. A context builder forms the system prompt, Gemini generates a response, validates it, and returns the final answer to the user or an error message.

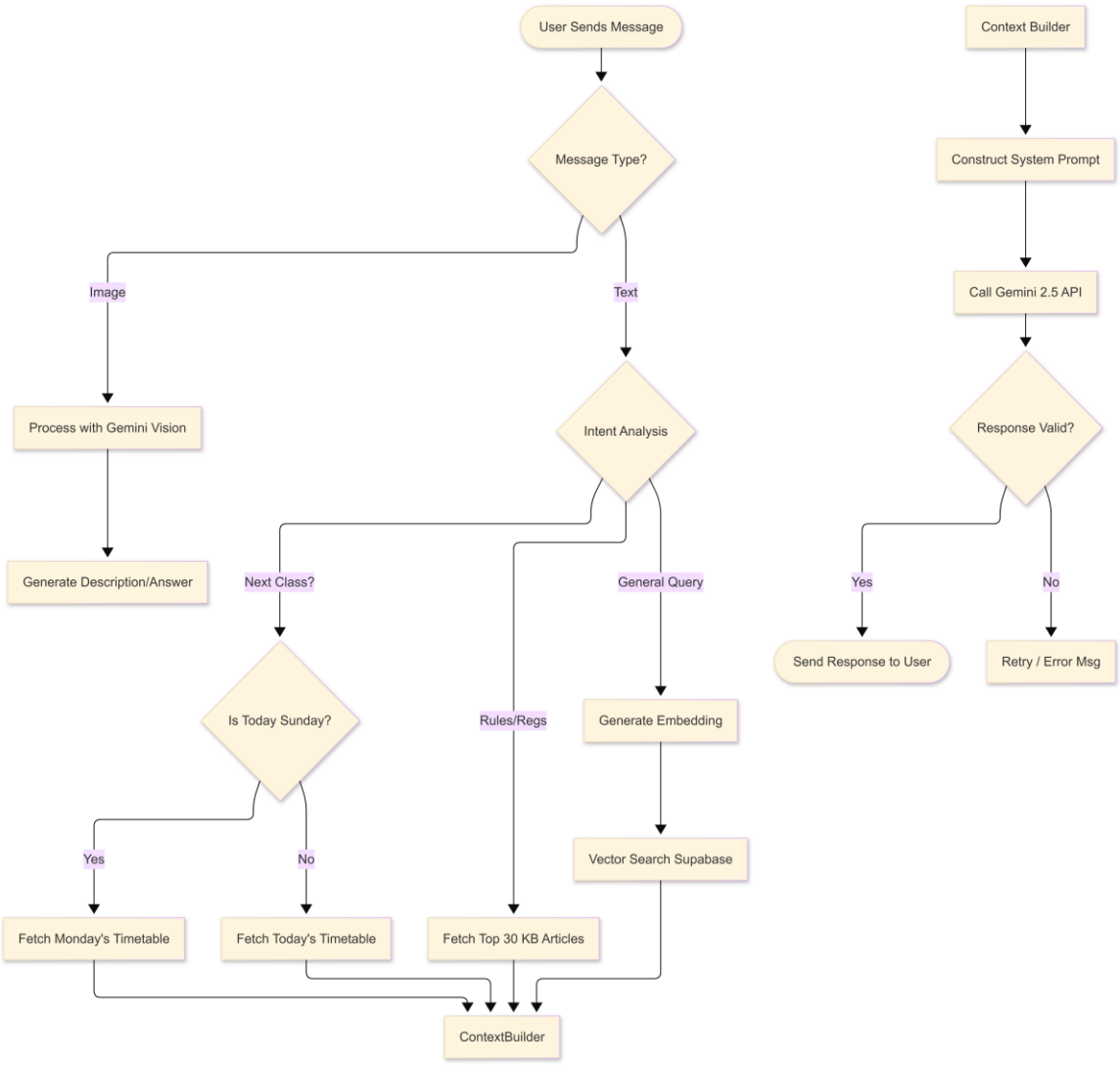


Fig.5.1.3 Activity diagram

UseCase Diagram:

This diagram shows the Campus Assistant ecosystem with role-based access for Admin, Faculty, and Students. Admin manages the global timetable, AI knowledge base, and user

roles. Faculty can view department timetables, dashboards, and post articles. Students access personal timetables, events, notices, virtual campus tours, and chat with Gemini AI. All users authenticate via Supabase. The system centralizes academic management, information sharing, and AI-powered assistance within a unified platform.

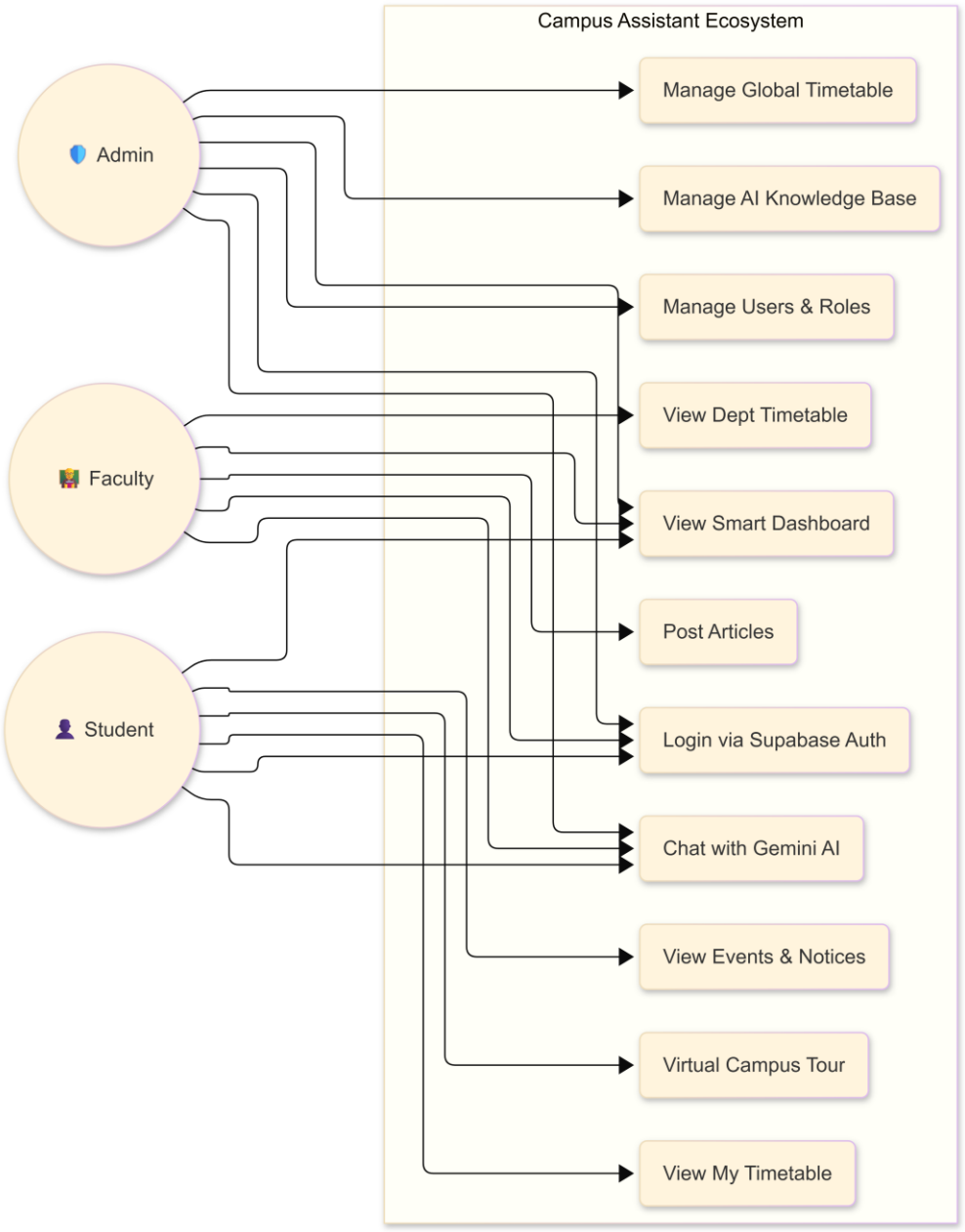


Fig.5.1.4 UseCase Diagram

Deployment Diagram:

This diagram shows the backend infrastructure of the Campus Assistant system. A Flutter client app communicates via HTTPS/REST with a Node.js Express API, using Hive for local storage. The backend integrates with the Supabase platform for authentication, object storage, and PostgreSQL database management. The database uses the pgvector extension to support vector similarity search. Google Gemini AI services are accessed securely over HTTPS for AI features. Supabase manages services and data flow, enabling scalable authentication, storage, AI processing, and efficient handling of application requests.

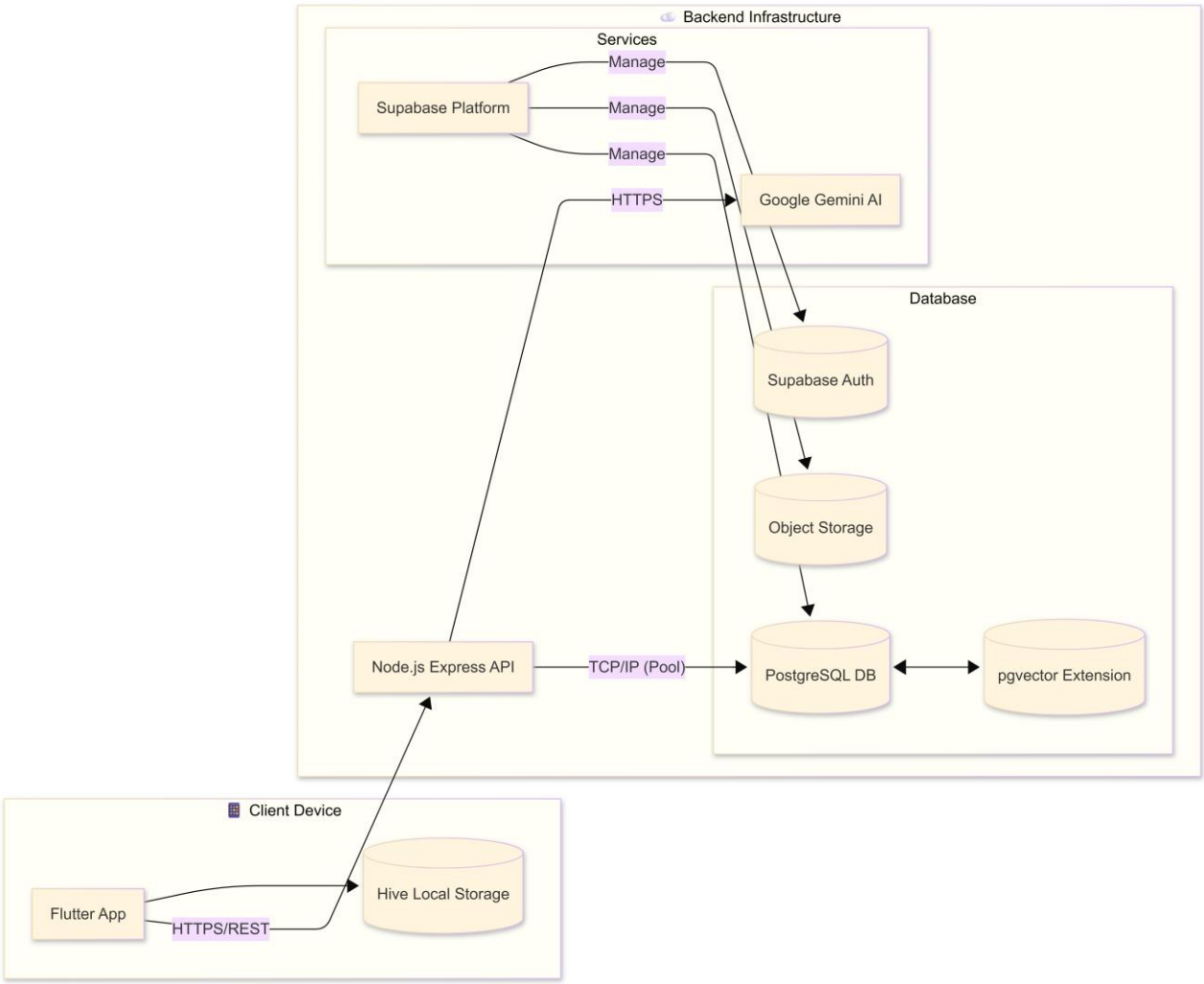


Fig.5.1.5 Deployment Diagram

5.2 SYSTEM DESIGN

5.2.1 SOFTWARE REQUIREMENTS

a) Frontend:

- **Flutter SDK:** For developing cross-platform mobile applications on Android and iOS.
- **Dart Language:** Programming language for Flutter application development.
- **Flutter Packages:**
    - http / dio for API requests
    - provider or riverpod for state management
    - flutter_chat_ui for chat interface
    - image_picker and speech_to_text for multimodal input (image & voice)

b) Backend:

- **Node.js (v16 or higher):** Server-side runtime for handling requests and API orchestration.
- **Express.js:** Web framework for building REST APIs and middleware handling.
- **Supabase:** Cloud backend providing authentication, database (PostgreSQL), and real-time data handling.
- **pgvector:** Extension for PostgreSQL to enable semantic vector storage and similarity search.
- **JWT (JSON Web Token):** Authentication and secure API access.

c) AI and Machine Learning:

- **Google Gemini API / LLM API:** For text, voice, and image-based AI processing.
- **Retrieval-Augmented Generation (RAG) Frameworks:** Integration with vectorized institutional knowledge for context-aware responses.
- **Node.js AI SDKs / REST API Integration:** To communicate with AI services.

d) Database:

- **PostgreSQL:** Relational database for storing user profiles, timetables, schedules, and administrative data.
- **Supabase Row-Level Security (RLS):** For enforcing secure role-based access control.

e) Development Tools:

- **VS Code / IntelliJ IDEA:** IDE for Flutter and Node.js development.
- **Postman:** API testing and debugging.
- **Git & GitHub / GitLab:** Version control.

f) Operating Systems:

- **Windows / macOS / Linux:** For development environments.
- **Android & iOS:** Target platforms for the Flutter mobile application.

5.2.1 HARDWARE REQUIREMENTS

a) Development Machines:

- **Processor:** Intel i5 or AMD Ryzen 5 (minimum)
- **RAM:** 8 GB (16 GB recommended for faster builds and AI testing)
- **Storage:** 256 GB SSD minimum (512 GB recommended for storing datasets, project files, and local databases)
- **Graphics:** Integrated GPU sufficient for Flutter UI rendering; discrete GPU optional for AI/ML testing

b) Server / Deployment Environment:

- **Cloud Hosting:** Free-tier or paid-tier cloud services for Node.js backend and Supabase hosting.
- **Processor:** Multi-core virtual CPU (vCPU) for handling concurrent requests.
- **RAM:** Minimum 2 GB (4 GB recommended for smooth AI query handling)

- **Storage:** Minimum 10 GB, scalable for institutional document storage and vectorized datasets.
- **Internet Connection:** Stable broadband (minimum 10 Mbps upload/download) for real-time AI API communication.

c) Client Devices (for End-Users):

- **Smartphones / Tablets:** Android 8.0+ or iOS 12+
- **Screen Resolution:** Minimum 720p for optimal mobile UI rendering
- **Internet Connectivity:** Stable Wi-Fi or mobile data (3G/4G/5G) for cloud-based AI and database access

CHAPTER 6

CODE

6.1 Source Code

```
/
├── flutter_app/          # Mobile Frontend
│   ├── lib/
│   │   ├── main.dart
│   │   ├── screens/      # Dashboard, Chat, Profile
│   │   ├── services/     # API, Auth
│   │   └── widgets/      # UI Components
│
└── backend/              # Node.js API
    ├── src/
    │   ├── index.ts
    │   ├── controllers/  # Logic
    │   └── services/     # Integrations
```

6.1.1 AI Service

```
import { GoogleGenerativeAI } from '@google/generative-ai';
import dotenv from 'dotenv';
import path from 'path';

const envPath = path.resolve(__dirname, '../../.env');
dotenv.config({ path: envPath });

const apiKeys = [
   process.env.GEMINI_API_KEY,
   process.env.GEMINI_API_KEY_1,
   process.env.GEMINI_API_KEY_2,
   process.env.GEMINI_API_KEY_3,
   process.env.GEMINI_API_KEY_4,
   process.env.GEMINI_API_KEY_5,
   process.env.GEMINI_API_KEY_6,
   process.env.GEMINI_API_KEY_7
].filter(k => k) as string[];
```

```
if (apiKeys.length === 0) {
   console.warn(`No GEMINI_API_KEYS found.`);
} else {
   const foundKeys = apiKeys.map(k => `...${k.substring(k.length - 4)}`);
   console.log(`Found ${apiKeys.length} Gemini API Key(s): ${foundKeys.join(', ')}`);
}

// Helper to execute AI calls with key rotation (Load Balancing + Failover)
const executeWithRetry = async <T>(operation: (genAI: GoogleGenerativeAI) => Promise<T>):
Promise<T> => {
   let lastError: any;
   const shuffledKeys = [...apiKeys].sort(() => Math.random() - 0.5);

   for (const key of shuffledKeys) {
      try {
         const genAI = new GoogleGenerativeAI(key);
         return await operation(genAI);
      } catch (error: any) {
         console.warn(` Error with key ...${key.substring(key.length - 4)}: ${error.message}`);
         lastError = error;
      }
   }
   throw lastError || new Error("All API keys failed");
};

export const generateText = async (prompt: string, context?: string) => {
   return executeWithRetry(async (genAI) => {
      console.log(" Generating text with gemini-2.5-flash...");
      const model = genAI.getGenerativeModel({ model: "gemini-2.5-flash" });
      const fullPrompt = context ? `Context: ${context}\n\nQuestion: ${prompt}` : prompt;
      const result = await model.generateContent(fullPrompt);
      return result.response.text();
   });
```

```
};

export const getEmbedding = async (text: string) => {
   return executeWithRetry(async (genAI) => {
      console.log(" Generating embedding with gemini-embedding-001...");
      const model = genAI.getGenerativeModel({ model: "gemini-embedding-001" });
      const result = await model.embedContent(text);
      return result.embedding.values;
   });
}
```

6.1.2 Chat Controller

```
import { Request, Response } from 'express';
import { generateText } from '../services/aiService';
import { supabase } from '../services/supabaseClient';
import { getEmbedding } from '../services/aiService';
import { WithAuthProp } from '@clerk/clerk-sdk-node';

export const handleTextChat = async (req: Request, res: Response): Promise<void> => {
   try {
      const userId = (req as WithAuthProp<Request>).auth.userId;
      let { message, conversationId, history } = req.body;
      conversationId = conversationId || 'ephemeral-session';

      // 1. Fetch User Profile
      const { data: profile } = await supabase
         .from('profiles')
         .select('role, department, year, section')
         .eq('id', userId)
         .single();

      // 2. Determine Context (Day of Week)
      const days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday'];
      const currentDay = days[new Date().getDay()];
      let targetDay = currentDay;
```

```
    if (message.toLowerCase().includes('tomorrow')) {
       targetDay = days[(new Date().getDay() + 1) % 7];
    }

    // 3. Fetch Timetable Context
    let timetableQuery = supabase.from('timetables').select('*');
    if (profile?.role === 'student') {
        timetableQuery = timetableQuery
          .eq('department', profile.department)
          .eq('year', profile.year)
          .eq('section', profile.section)
          .eq('day_of_week', targetDay);
    }

    const [timetableRes, remindersRes] = await Promise.all([
       timetableQuery,
       supabase.from('reminders').select('*').eq('user_id', userId).eq('is_completed', false)
    ]);

    const timetables = timetableRes.data || [];
    const reminders = remindersRes.data || [];

    // 4. Fetch Knowledge Base Context (RAG)
    const embedding = await getEmbedding(message);
    const { data: kbData } = await supabase.rpc('match_kb_articles', {
       query_embedding: embedding,
       match_threshold: 0.3,
       match_count: 3
    });

    // 5. Construct System Prompt
    const systemContext = `
You are the Campus Assistant AI. You are talking to a ${profile?.role || 'User'}.
```

```
Answer the user's question based on their role.

--- USER CONTEXT ---
TIMETABLE (Showing Data For: ${targetDay}):
${timetables.length ? timetables.map((t: any) => `- ${t.course_name} at ${t.start_time} [Loc: ${t.location}]`).join('\n') : `No classes.`}

PENDING REMINDERS:
${reminders.length ? reminders.map((r: any) => `- ${r.title} (Due: ${r.due_at})`).join('\n') : "No pending reminders."}

KNOWLEDGE BASE:
${kbData && kbData.length > 0 ? kbData.map((d: any) => `- ${d.title}: ${d.content}`).join('\n') : "No specific KB articles found."}
--------------------
`;
      const responseText = await generateText(message, systemContext);

      res.json({
         response: responseText,
         conversationId: conversationId
      });
   } catch (error: any) {
      console.error(error);
      res.status(500).json({ error: error.message || 'Failed to generate response' });
   }
}
```

6.1.3 Dashboard Screen

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:go_router/go_router.dart';
import '../providers/auth_provider.dart';
import '../services/api_service.dart';
```

```
class DashboardScreen extends StatefulWidget {
  const DashboardScreen({super.key});

  @override
  State<DashboardScreen> createState() => _DashboardScreenState();
}

class _DashboardScreenState extends State<DashboardScreen> {
  final ApiService _apiService = ApiService();
  Map<String, dynamic>? _dashboardData;
  bool _isLoading = true;

  @override
  void initState() {
    super.initState();
    _loadDashboardData();
  }

  Future<void> _loadDashboardData() async {
    setState(() => _isLoading = true);
    try {
      final data = await _apiService.get('/api/dashboard');
      setState(() {
        _dashboardData = data;
        _isLoading = false;
      });
    } catch (e) {
      if (mounted) setState(() => _isLoading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    final authProvider = Provider.of<AuthProvider>(context);
```

```
final user = authProvider.user;

return Container(
  width: double.infinity,
  height: double.infinity,
  decoration: const BoxDecoration(
    gradient: LinearGradient(colors: [Color(0xFF1A1A2E), Color(0xFF16213E)]),
  ),
  child: SafeArea(
    child: _isLoading
        ? const Center(child: CircularProgressIndicator())
        : SingleChildScrollView(
            padding: const EdgeInsets.all(24.0),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  'Welcome back, ${user?.fullName ?? 'Student'} ',
                  style: const TextStyle(fontSize: 28, fontWeight: FontWeight.bold, color: Colors.white),
                ),
                const SizedBox(height: 32),

                // Stats / Next Class
                _buildNextClassCard(),
                const SizedBox(height: 24),

                // Feature Grid
                _buildVirtualTourCard(),
                const SizedBox(height: 24),
                _buildRemindersCard(),
              ],
            ),
          ),
  ),
```

```
    );
  }

  Widget _buildNextClassCard() {
    final nextClass = _dashboardData?['nextClass'];
    if (nextClass == null) return const SizedBox.shrink();

    return Container(
      padding: const EdgeInsets.all(24),
      decoration: BoxDecoration(
        color: Colors.white.withOpacity(0.05),
        borderRadius: BorderRadius.circular(24),
        border: Border.all(color: Colors.white.withOpacity(0.1)),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
                  const Text('UP NEXT', style: TextStyle(color: Colors.blueAccent, fontWeight:
FontWeight.bold)),
          const SizedBox(height: 12),
          Text(
            nextClass['course_name'],
            style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold, color: Colors.white),
          ),
          Text(
            '${nextClass['start_time']} • ${nextClass['location']}',
            style: TextStyle(color: Colors.white.withOpacity(0.7)),
          ),
        ],
      ),
    );
  }
}
```

6.1.4 API Service

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../config/env.dart';

class ApiService {
  static final ApiService _instance = ApiService._internal();
  factory ApiService() => _instance;
  ApiService._internal();

  String? _authToken;

  String get baseUrl => EnvConfig.apiUrl;

  Map<String, String> _getHeaders({bool includeAuth = true}) {
    final headers = {'Content-Type': 'application/json'};
    if (includeAuth && _authToken != null) {
      headers['Authorization'] = 'Bearer $_authToken';
    }
    return headers;
  }

  Future<dynamic> get(String endpoint) async {
    try {
      final response = await http.get(
        Uri.parse('$baseUrl$endpoint'),
        headers: _getHeaders(),
      );
      return _handleResponse(response);
    } catch (e) {
      throw Exception('Network error: $e');
    }
  }
```

```
  Future<dynamic> post(String endpoint, dynamic body) async {
    try {
      final response = await http.post(
        Uri.parse('$baseUrl$endpoint'),
        headers: _getHeaders(),
        body: jsonEncode(body),
      );
      return _handleResponse(response);
    } catch (e) {
      throw Exception('Network error: $e');
    }
  }

  dynamic _handleResponse(http.Response response) {
    if (response.statusCode >= 200 && response.statusCode < 300) {
      return jsonDecode(response.body);
    } else {
      throw Exception('Request failed with status ${response.statusCode}');
    }
  }
}


    == '__main__':
```

CHAPTER 7

RESULTS AND DISCUSSIONS

7.1 RESULT

The Proactive Multimodal Academic Support System (PMASS) was implemented and tested across multiple scenarios involving students, faculty, and administrative users. The results demonstrate the effectiveness of the system in providing personalized, context-aware, and multimodal academic assistance while maintaining secure and reliable operations.

1. Student Module Results

- **Context-Aware Responses:** AI chat accurately retrieved student-specific timetable information, upcoming deadlines, and course prerequisites.
- **Multimodal Interaction:** Students successfully submitted queries via text, voice, and images. Responses were contextually accurate and delivered in real-time.
- **Task Management and Planning:** Personalized study plans and task reminders were generated based on course schedules and deadlines.
- **Virtual Campus Navigation:** Students could access building locations and navigation guidance through the mobile interface.

2. Faculty Module Results

- **AI Lecture Assistance:** Faculty received suggested lecture outlines, verified resource links, and tone-customized content for lectures.
- **Student Monitoring:** Faculty dashboards displayed student engagement and performance trends, facilitating timely academic interventions.
- **Content Review:** Faculty were able to approve or edit AI-generated academic suggestions, ensuring oversight and maintaining academic integrity.

3. Administrative Module Results

- **Centralized Timetable Management:** Bulk import of schedules and management of institutional data was seamless.
- **Role-Based Access Control:** RBAC and Row-Level Security effectively restricted data access based on user roles.
- **System Monitoring:** Administrators could track AI usage, system health, and database activity through dashboards.

4. System Performance Results

- **Resilience:** Multi-key AI load balancing prevented service downtime due to API rate limits, maintaining continuous service even during peak loads.
- **Reduced Latency:** Heartbeat keep-alive mechanisms minimized cold-start delays in serverless deployments, providing fast response times.
- **Accuracy:** Retrieval-Augmented Generation (RAG) integration with vectorized institutional documents significantly reduced AI hallucinations, improving factual correctness and relevance of responses.
- **User Satisfaction:** Testing with students and faculty indicated high usability and positive feedback for the proactive AI assistance and seamless interface.

Admin Interface Overview

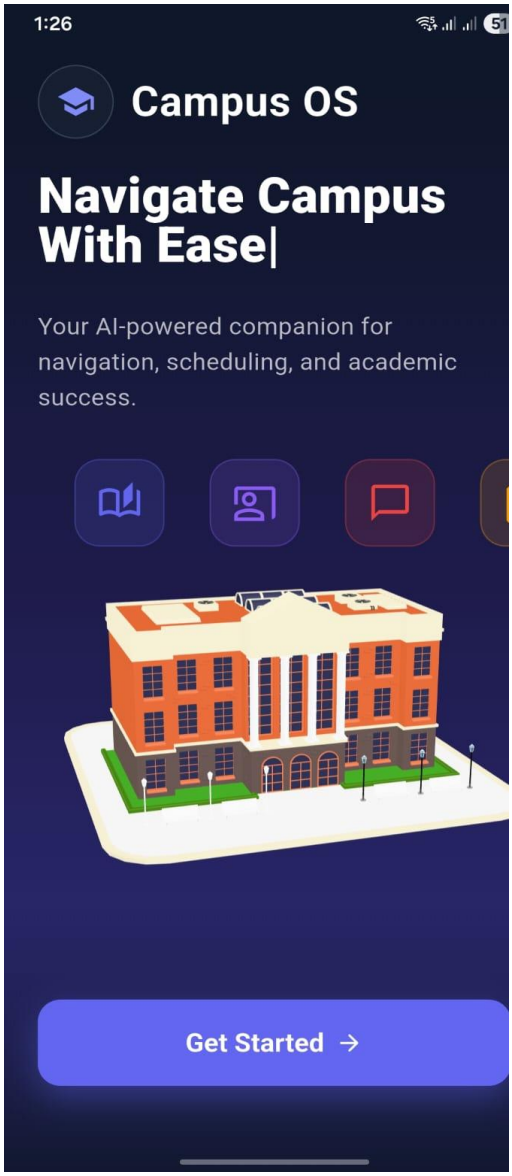


Fig 7.1.1 Landing Page (App Homepage)

The Campus OS landing page introduces the app as an AI-powered campus companion for navigation, scheduling, and academic support. A clear "Get Started" button enables users to quickly begin using the system.

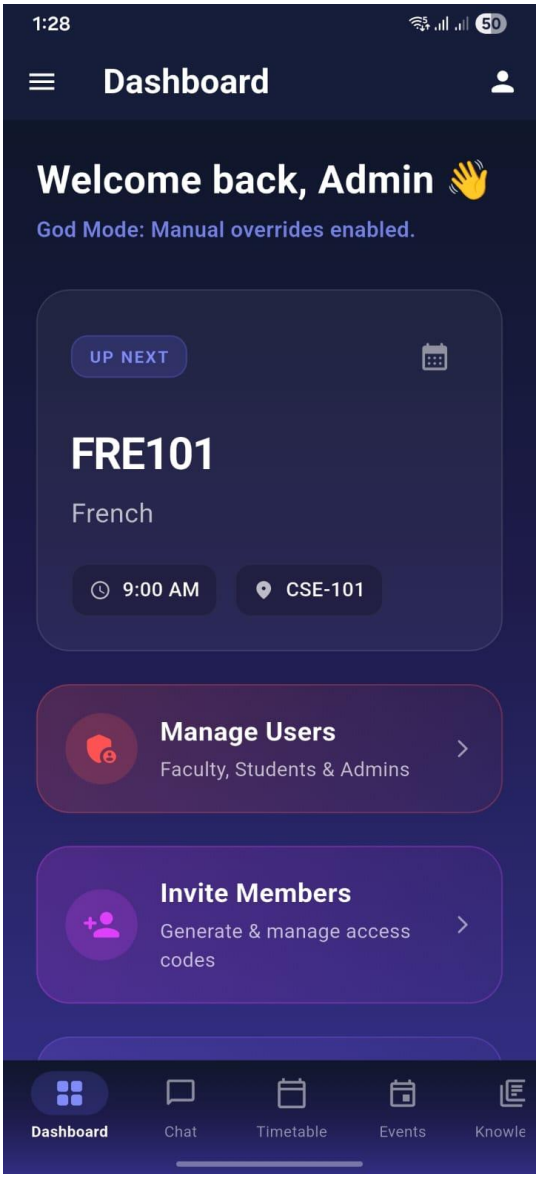


Fig 7.1.2 Admin Dashboard (God Mode Interface)

The Admin Dashboard provides high-privilege controls, displaying upcoming schedules and administrative tools such as user management and member invitations.

"God Mode" indicates enabled manual overrides, allowing administrators to perform critical system-level operations securely.

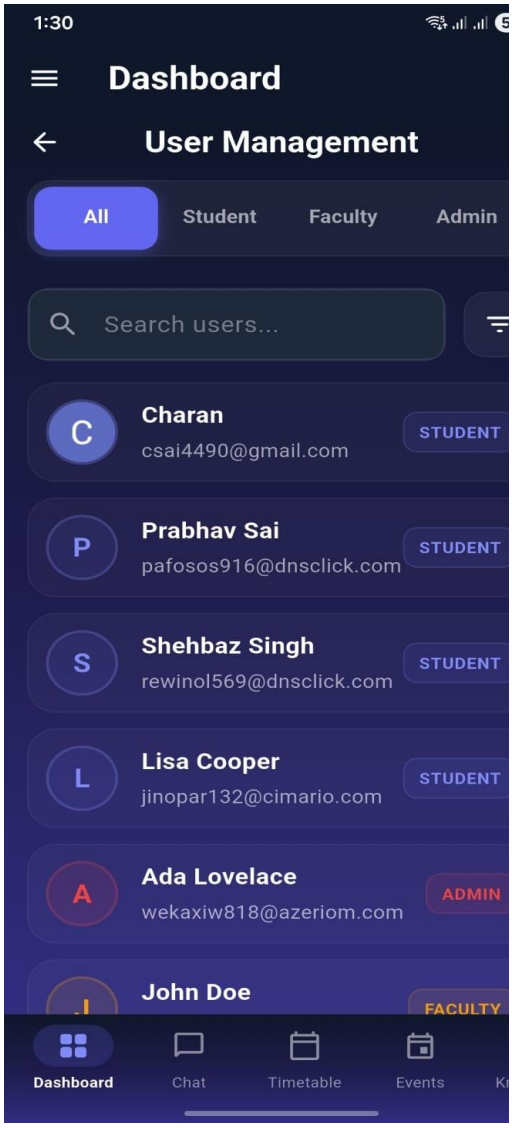


Fig 7.1.3: User Management Interface

The User Management screen enables administrators to view, search, and filter users by roles such as Student, Faculty, and Admin.

It provides centralized control for monitoring accounts and managing role-based access within the system.

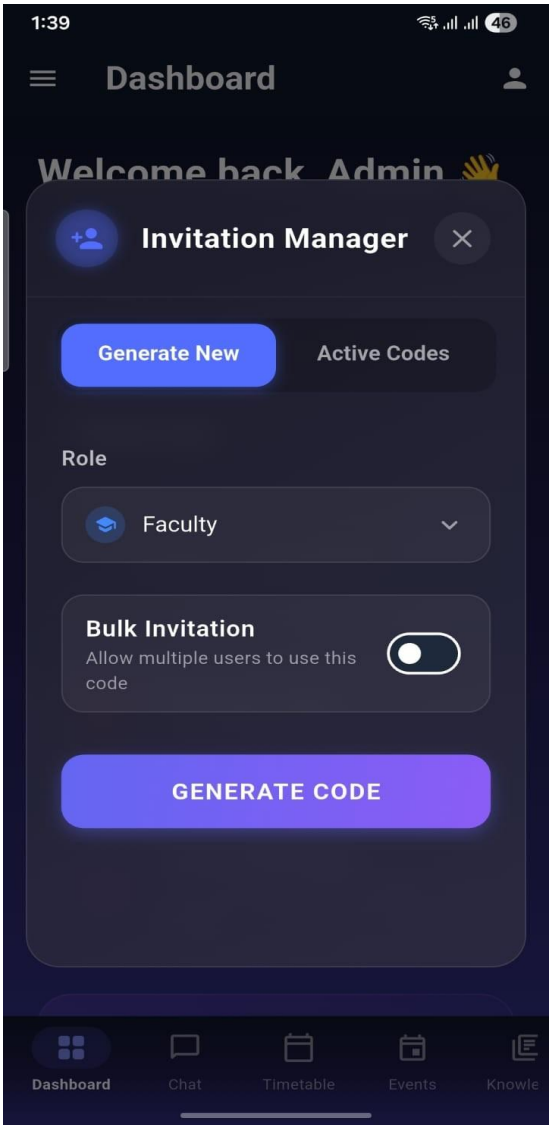


Fig 7.1.4: Invitation Manager Interface

The Invitation Manager allows administrators to generate secure access codes for role-based onboarding, such as Faculty or Admin accounts. It supports single-use or bulk invitations, enabling controlled user promotion and streamlined institutional access management.

Faculty Interface Overview

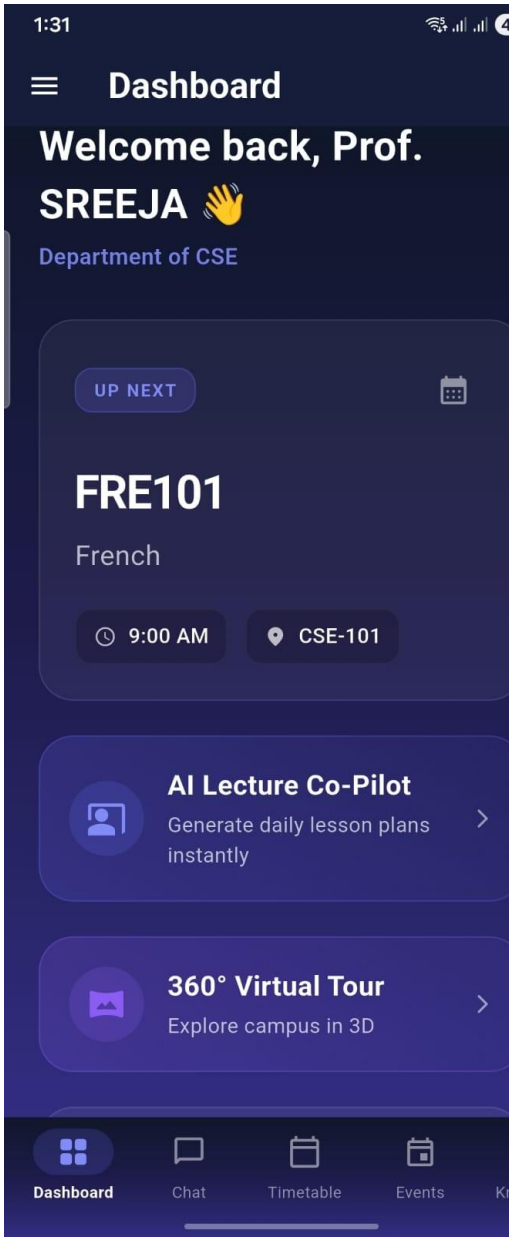


Fig 7.1.5: Faculty Dashboard Interface

The Faculty Dashboard provides personalized academic insights, including upcoming classes, department details, and quick access to teaching tools. It integrates features such as the AI Lecture Co-Pilot and 360° Virtual Tour to support instructional planning and campus navigation.

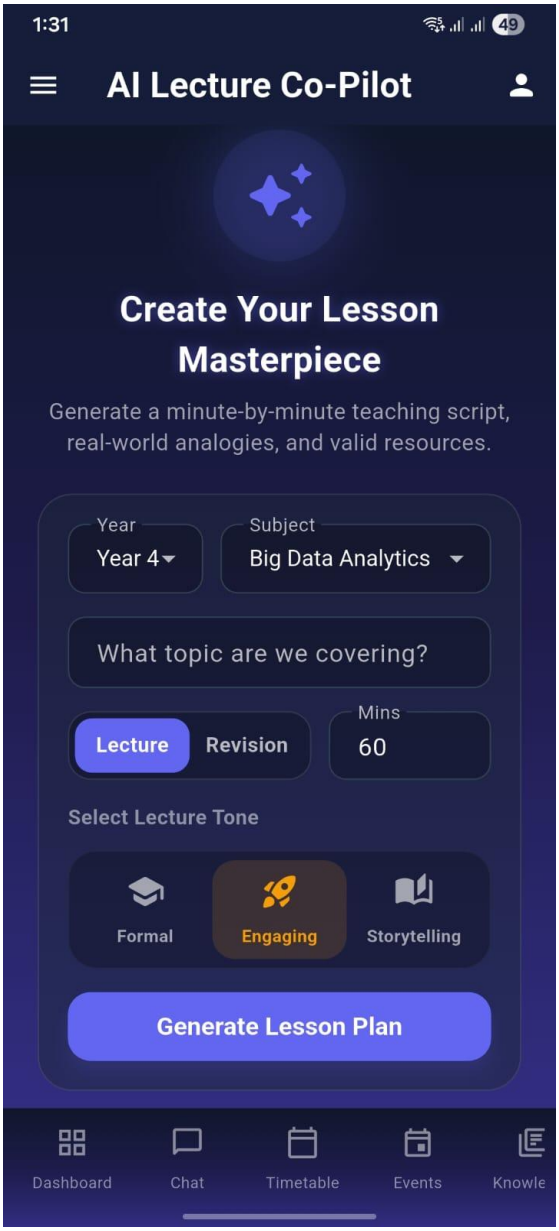


Fig 7.1.6: AI Lecture Co-Pilot Interface

The AI Lecture Co-Pilot enables faculty to generate structured lesson plans by selecting academic parameters such as year, subject, topic, duration, and lecture mode. It also allows tone customization (Formal, Engaging, Storytelling) to produce pedagogically aligned teaching scripts and resources.

Fig 7.1.7: Generated Lecture Script View

This screen displays the AI-generated expert lecture script, including an engaging hook and structured content for the selected topic (CNNs). It allows faculty to review, download, or access supporting links for delivering a well-prepared session.

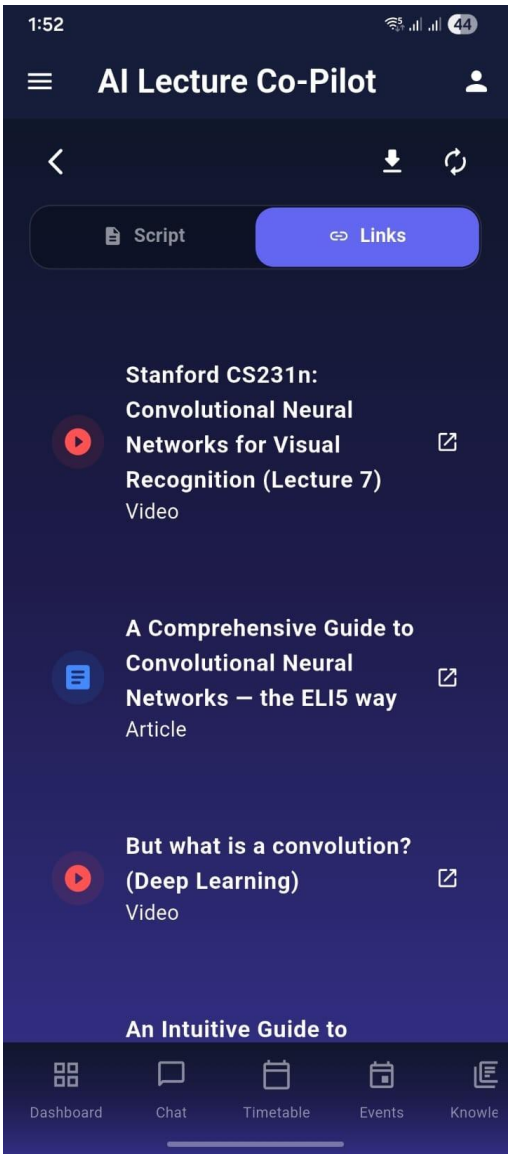


Fig 7.1.8: Generated Lecture Links View

This screen displays curated supporting resources for the selected topic (CNNs), including recommended videos and articles. It allows faculty to quickly access high-quality external materials to enhance their lecture delivery. Users can open links in a new tab, download resources, or switch back to the script view for structured lecture content, ensuring a well-supported and comprehensive teaching session.

Student Interface Overview

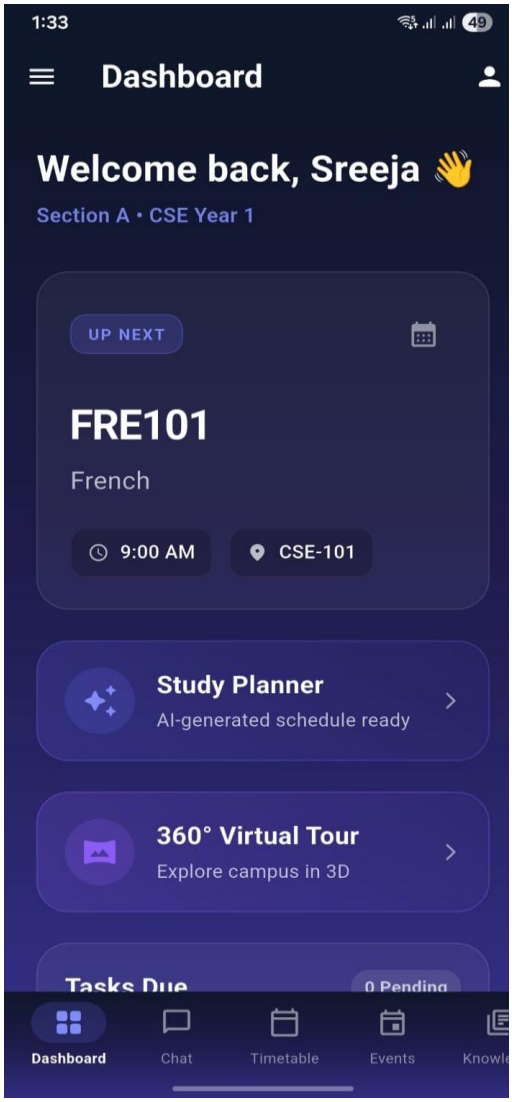
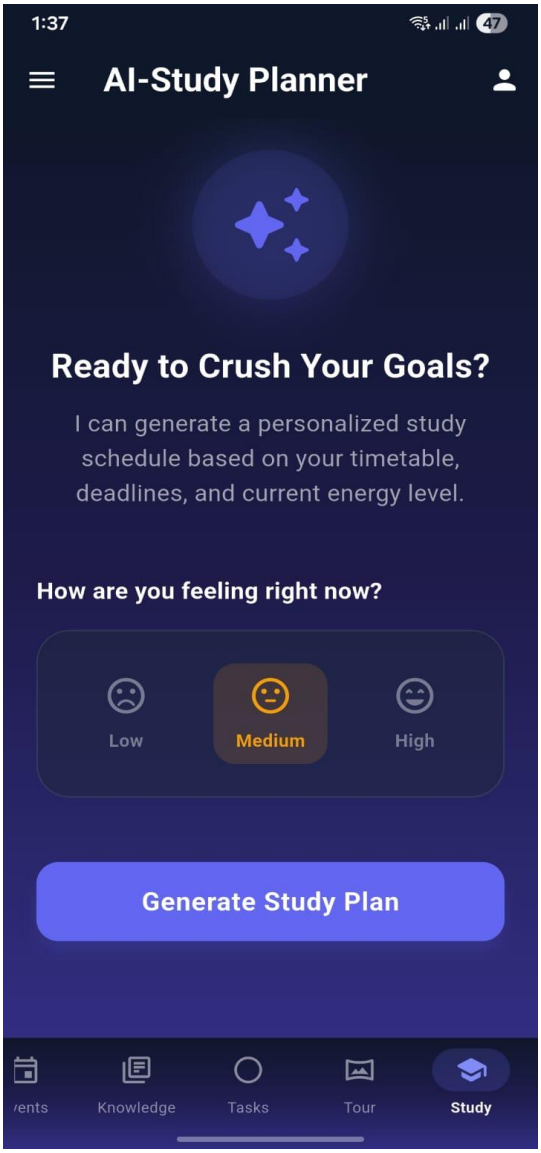


Fig 7.1.9: Student Dashboard View

This screen presents the personalized dashboard of the application, welcoming the user and displaying their academic details such as section and year. The dashboard also provides quick access to key features including the AI-generated Study Planner, 360° Virtual Campus Tour, and pending tasks. A bottom navigation bar enables seamless movement across core modules like Dashboard, Chat, Timetable,Events, and Knowledge for efficient daily academic management.



7.1.10: AI Study Planner View

This screen presents the AI-powered Study Planner feature, designed to generate a personalized study schedule based on the user's timetable, deadlines, and current energy level. The interface prompts users to select how they are feeling (Low, Medium, or High), enabling the system to adapt the study intensity accordingly.

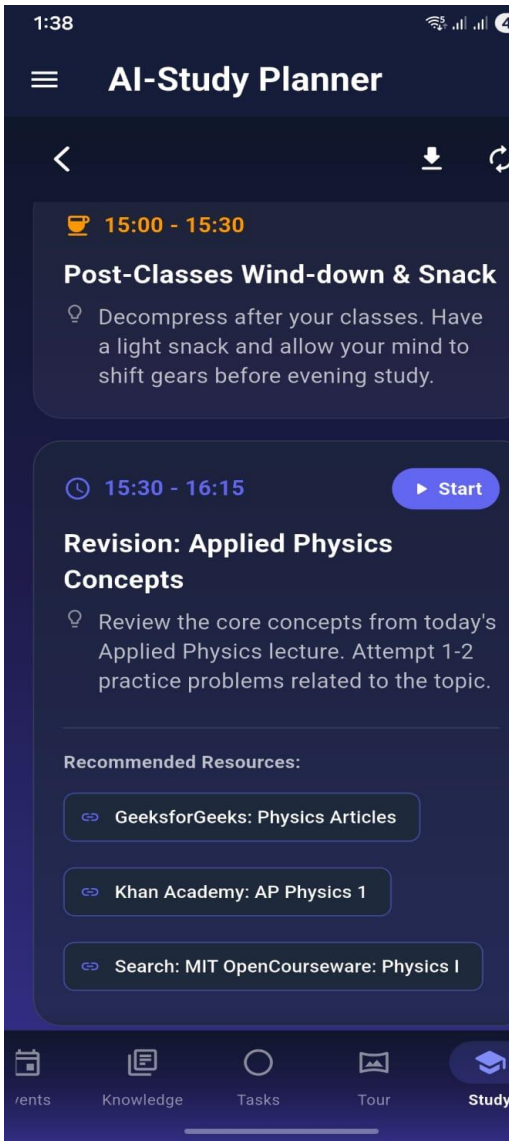


Fig 7.1.11: Generated Study Plan View

This screen displays the AI-generated personalized study schedule, structured into time-based activity blocks. It includes a balanced mix of relaxation and focused revision sessions tailored to the user's selected energy level

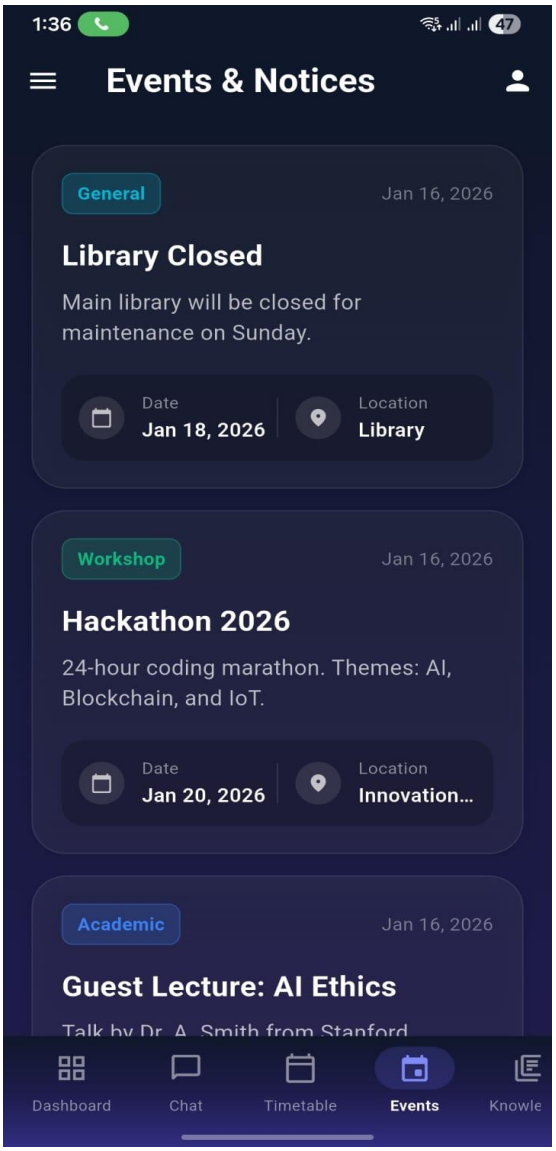


Fig 7.1.12: Events & Notices View

This screen displays the Events & Notices section, providing users with important campus announcements and upcoming activities. Each event card includes a category label (e.g., General, Workshop, Academic), event title, brief description, date, and location details.

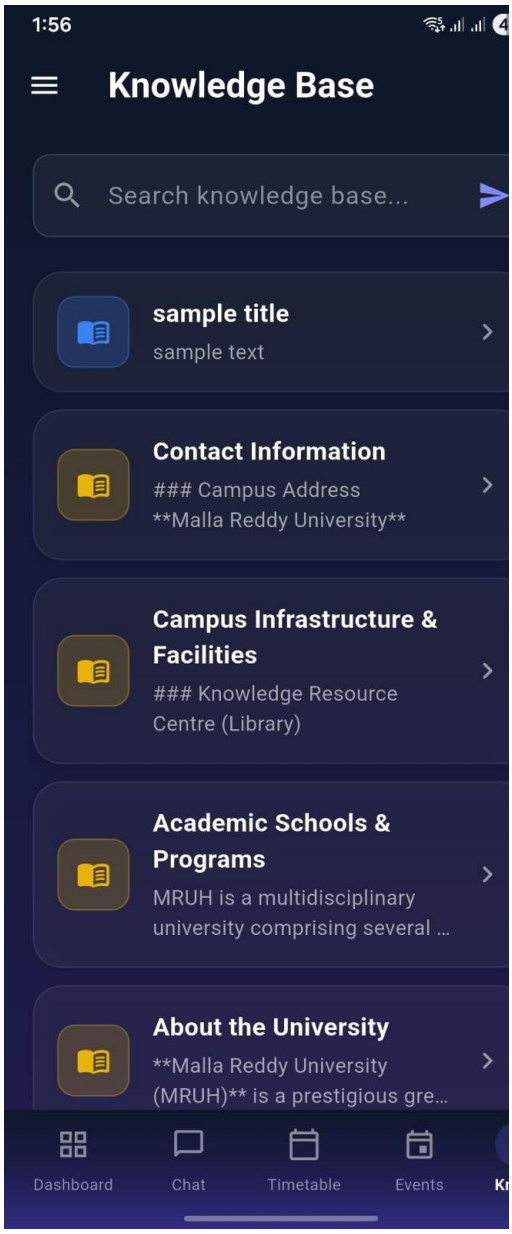


Fig 7.1.13: Knowledge Base View

This screen displays the Knowledge Base section, which provides structured and searchable institutional information for students. Users can utilize the search bar to quickly find relevant content. The section includes categorized entries such as Contact Information, Campus Infrastructure & Facilities, Academic Schools & Programs, and About the University.

Fig 7.1.14: Tasks View

This screen displays the Tasks module, which helps users manage and track their pending academic activities. It shows a list of tasks with details such as task title, category (e.g., Assignment), and due date. Users can mark tasks as completed using the checkbox or delete them using the delete icon.

Fig 7.1.15: Campus 360° Tour View

This screen presents the Campus 360° Tour feature, enabling users to virtually explore university locations through an immersive panoramic view. Users can browse campus areas by category (e.g., Campus View, VC Chamber, Admissions) using the dropdown filters.

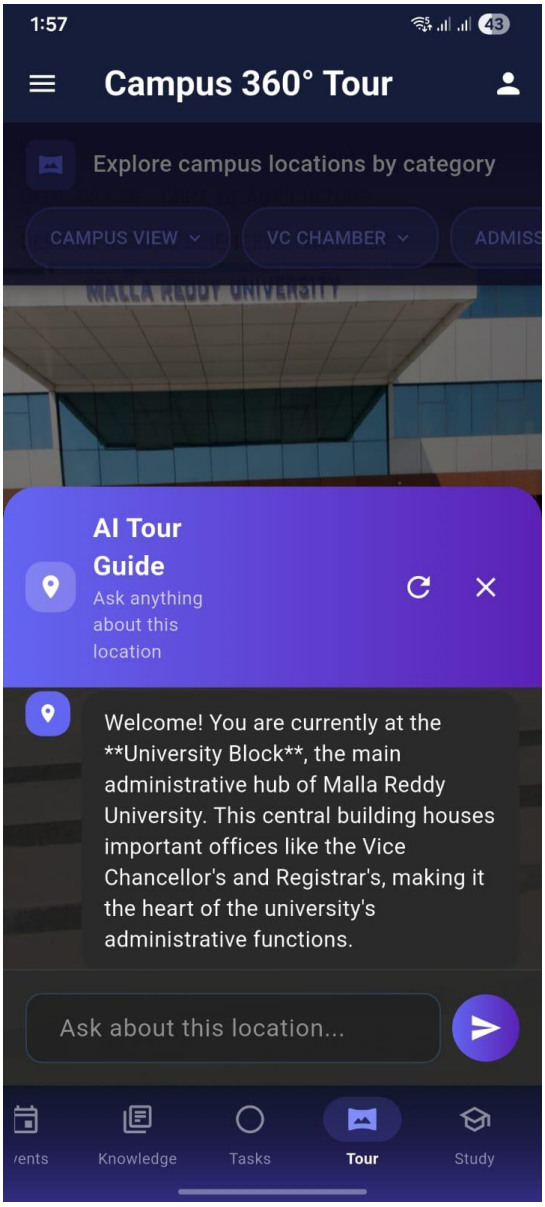


Fig 7.1.16: AI Tour Guide Interaction Screen

This screen shows the AI Tour Guide within the Campus 360° Tour, providing interactive information about the selected campus location. Users can ask questions about the University Block and navigate easily using the category filters and bottom menu.

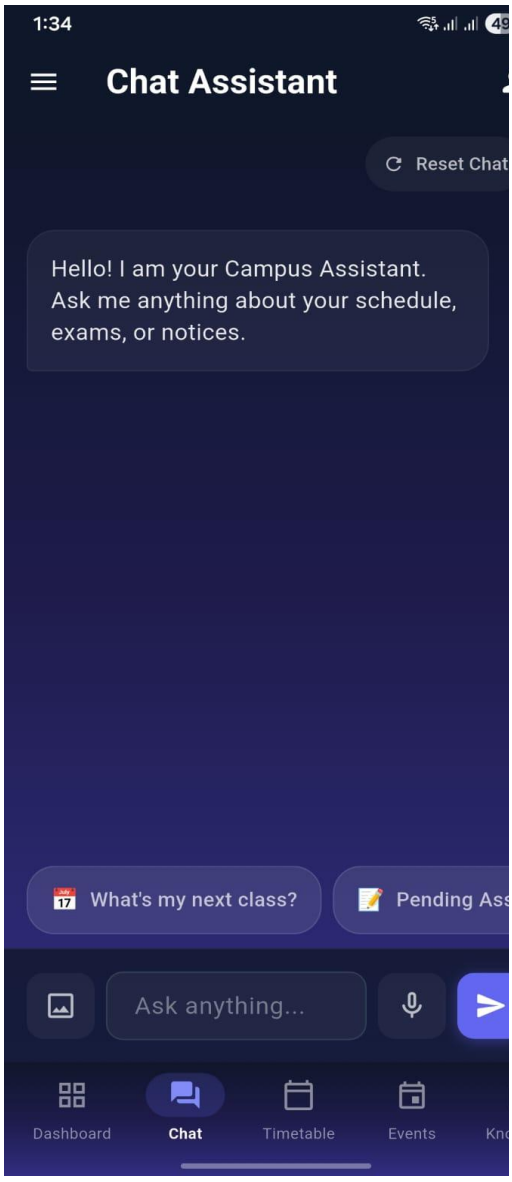


Fig 7.1.17: Chat Assistant Interface View

This screen shows the Chat Assistant feature that allows users to ask questions about schedules, exams, and campus notices through an interactive chat interface. It also provides quick-action buttons, voice input, and navigation tabs for easy access to other sections.

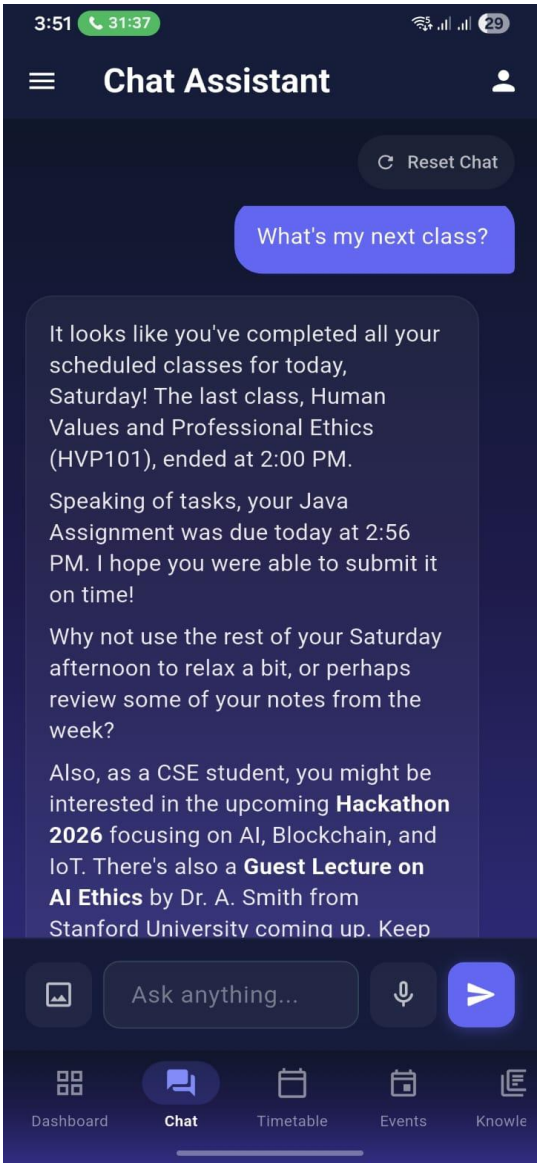
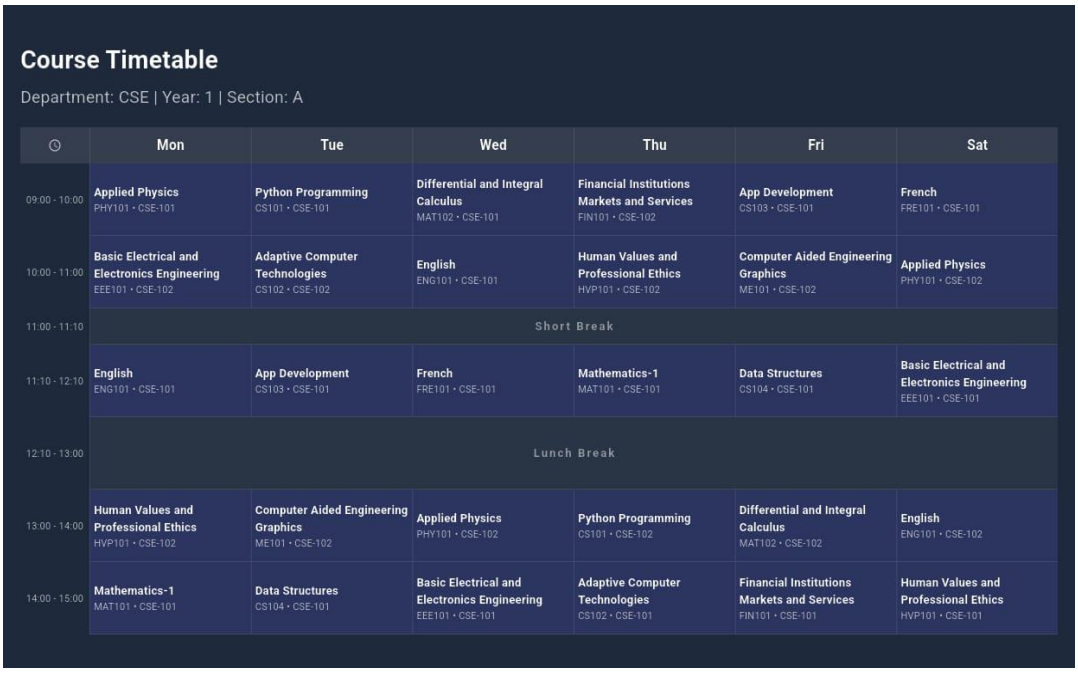


Fig 7.1.18: Chat Assistance

This screen demonstrates the AI chat interface responding to a student prompt, "What's my next class?". The system retrieves context-aware information from the student's timetable and displays the upcoming class with relevant details, illustrating personalized academic assistance.



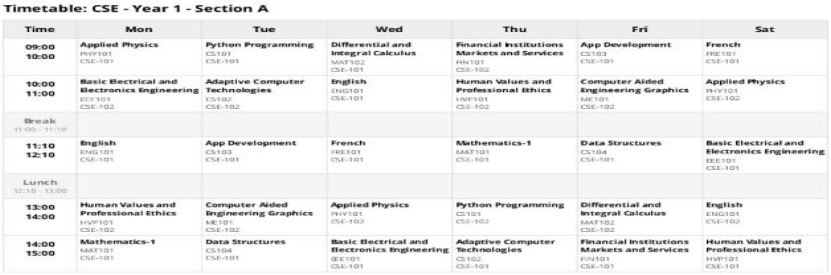
Course Timetable

Department: CSE | Year: 1 | Section: A

	Mon	Tue	Wed	Thu	Fri	Sat
09:00 - 10:00	Applied Physics PHY101 • CSE-101	Python Programming CS101 • CSE-101	Differential and Integral Calculus MAT102 • CSE-101	Financial Institutions Markets and Services FIN101 • CSE-102	App Development CS103 • CSE-101	French FRE101 • CSE-101
10:00 - 11:00	Basic Electrical and Electronics Engineering EEE101 • CSE-102	Adaptive Computer Technologies CS102 • CSE-102	English ENG101 • CSE-101	Human Values and Professional Ethics HVP101 • CSE-102	Computer Aided Engineering Graphics ME101 • CSE-102	Applied Physics PHY101 • CSE-102
11:00 - 11:10	Short Break					
11:10 - 12:10	English ENG101 • CSE-101	App Development CS103 • CSE-101	French FRE101 • CSE-101	Mathematics-1 MAT101 • CSE-101	Data Structures CS104 • CSE-101	Basic Electrical and Electronics Engineering EEE101 • CSE-101
12:10 - 13:00	Lunch Break					
13:00 - 14:00	Human Values and Professional Ethics HVP101 • CSE-102	Computer Aided Engineering Graphics ME101 • CSE-102	Applied Physics PHY101 • CSE-102	Python Programming CS101 • CSE-102	Differential and Integral Calculus MAT102 • CSE-102	English ENG101 • CSE-102
14:00 - 15:00	Mathematics-1 MAT101 • CSE-101	Data Structures CS104 • CSE-101	Basic Electrical and Electronics Engineering EEE101 • CSE-101	Adaptive Computer Technologies CS102 • CSE-101	Financial Institutions Markets and Services FIN101 • CSE-101	Human Values and Professional Ethics HVP101 • CSE-101

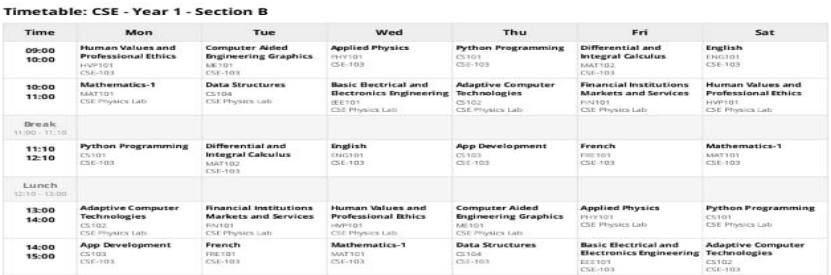
Fig 7.1.19: Course Timetable Screen

This screen displays the Course Timetable for CSE (Year 1, Section A), showing subject schedules from Monday to Saturday with time slots. It includes clearly marked short and lunch breaks, helping students easily track their daily classes.



Timetable: CSE - Year 1 - Section A

Time	Mon	Tue	Wed	Thu	Fri	Sat
09:00 10:00	Applied Physics PHY101 CSE-101	Python Programming CS101 CSE-101	Differential and Integral Calculus MAT102 CSE-101	Financial Institutions Markets and Services FIN101 CSE-102	App Development CS103 CSE-101	French FRE101 CSE-101
10:00 11:00	Basic Electrical and Electronics Engineering EEE101 CSE-102	Adaptive Computer Technologies CS102 CSE-102	English ENG101 CSE-101	Human Values and Professional Ethics HVP101 CSE-102	Computer Aided Engineering Graphics ME101 CSE-102	Applied Physics PHY101 CSE-102
Break 11:00 - 11:10						
11:10 12:10	English ENG101 CSE-101	App Development CS103 CSE-101	French FRE101 CSE-101	Mathematics-1 MAT101 CSE-101	Data Structures CS104 CSE-101	Basic Electrical and Electronics Engineering EEE101 CSE-101
Lunch 12:10 - 13:00						
13:00 14:00	Human Values and Professional Ethics HVP101 CSE-102	Computer Aided Engineering Graphics ME101 CSE-102	Applied Physics PHY101 CSE-102	Python Programming CS101 CSE-102	Differential and Integral Calculus MAT102 CSE-102	English ENG101 CSE-102
14:00 15:00	Mathematics-1 MAT101 CSE-101	Data Structures CS104 CSE-101	Basic Electrical and Electronics Engineering EEE101 CSE-101	Adaptive Computer Technologies CS102 CSE-101	Financial Institutions Markets and Services FIN101 CSE-101	Human Values and Professional Ethics HVP101 CSE-101



Timetable: CSE - Year 1 - Section B

Time	Mon	Tue	Wed	Thu	Fri	Sat
09:00 10:00	Human Values and Professional Ethics HVP101 CSE-103	Computer Aided Engineering Graphics ME101 CSE-103	Applied Physics PHY101 CSE-103	Python Programming CS101 CSE-103	Differential and Integral Calculus MAT102 CSE-103	English ENG101 CSE-103
10:00 11:00	Mathematics-1 MAT101 CSE Physics Lab	Data Structures CS104 CSE Physics Lab	Basic Electrical and Electronics Engineering EEE101 CSE Physics Lab	Adaptive Computer Technologies CS102 CSE Physics Lab	Financial Institutions Markets and Services FIN101 CSE Physics Lab	Human Values and Professional Ethics HVP101 CSE Physics Lab
Break 11:00 - 11:10						
11:10 12:10	Python Programming CS101 CSE-103	Differential and Integral Calculus MAT102 CSE-103	English ENG101 CSE-103	App Development CS103 CSE-103	French FRE101 CSE-103	Mathematics-1 MAT101 CSE-103
Lunch 12:10 - 13:00						
13:00 14:00	Adaptive Computer Technologies CS102 CSE Physics Lab	Financial Institutions Markets and Services FIN101 CSE Physics Lab	Human Values and Professional Ethics HVP101 CSE Physics Lab	Computer Aided Engineering Graphics ME101 CSE Physics Lab	Applied Physics PHY101 CSE Physics Lab	Python Programming CS101 CSE Physics Lab
14:00 15:00	App Development CS103 CSE-103	French FRE101 CSE-103	Mathematics-1 MAT101 CSE-103	Data Structures CS104 CSE-103	Basic Electrical and Electronics Engineering EEE101 CSE-103	Adaptive Computer Technologies CS102 CSE-103

Fig 7.1.20 Screenshot of the downloaded PDF

This screenshot shows timetable including selected subjects, time slots, and course codes. It provides a preview of the schedule before downloading for offline reference.

CHAPTER 8

CONCULSION

8.1 CONCLUSION

The Proactive Multimodal Academic Support System (PMASS) demonstrates a significant advancement in the digitization and intelligent management of university academic services. By unifying multiple campus functions—such as scheduling, advisory support, academic planning, and navigation—into a single mobile-first platform, PMASS reduces the complexity and fragmentation inherent in traditional digital ecosystems. The system's context-fusion architecture ensures that AI-generated responses are grounded in real-time, institution-specific data, resulting in highly personalized guidance for students. This approach addresses limitations in existing AI academic tools, which often provide generalized advice detached from actual institutional context.

Students experience enhanced learning support through multimodal AI interactions, including text, voice, and image inputs. Personalized study plans, reminders for assignments and deadlines, and interactive virtual campus navigation improve academic efficiency and engagement. Faculty members benefit from AI-assisted lecture preparation, curated resource suggestions, and real-time monitoring of student engagement, enabling more effective teaching and informed academic decisions. Administrative functions, including centralized timetable management, role-based access control, and activity monitoring dashboards, ensure that institutional governance, security, and compliance standards are strictly maintained.

The system's robust backend infrastructure, including multi-key AI load balancing and heartbeat keep-alive mechanisms, ensures operational reliability even under constrained deployment conditions such as free-tier cloud hosting. This guarantees uninterrupted AI assistance, mitigating latency issues caused by serverless cold starts or API rate-limit restrictions. Furthermore, integration of retrieval-augmented generation with vectorized institutional memory significantly improves the factual accuracy of AI responses, minimizing errors and increasing stakeholder trust in the system.

Security and governance have been prioritized through layered authentication, authorization,

and Row-Level Security, ensuring that sensitive student and institutional data remain protected while enabling role-specific access. Faculty oversight is preserved through human-in-the-loop mechanisms, where AI-generated suggestions can be reviewed or modified before dissemination, promoting accountability and ethical use of AI in education.

Overall, PMASS exemplifies how a proactive, multimodal, and secure AI-driven system can transform higher education support services. It enhances student engagement, optimizes faculty productivity, and strengthens administrative efficiency while maintaining trust, security, and compliance. The framework lays a foundation for continued innovation in intelligent academic assistance and demonstrates the potential for further expansion, including predictive analytics, deeper integration with learning management systems, and advanced multimodal AI capabilities to meet evolving educational needs.

8.2 FUTURE SCOPE

The future scope of the Proactive Multimodal Academic Support System (PMASS) includes enhancing its capabilities with more advanced AI models for deeper personalization and predictive academic guidance. The system can be extended to support additional features such as real-time analytics, automated career recommendations, and integration with external educational platforms. It can also be scaled to support multiple institutions, enabling a unified digital ecosystem across universities. Further improvements in offline functionality, multilingual support, and stronger data security mechanisms will make the system more accessible, reliable, and widely adoptable in diverse educational environments.

REFERENCES

1. Sherif Abdelhamid, James Bangura, Shahryar Shah, et al. 2025. *Advisely: AI Powered Academic Advising Using Large Language Models (LLMs)*. In Conference Proceedings, New Perspectives in Science Education 2025.
2. Google DeepMind. 2023. *Gemini: A Family of Highly Capable Multimodal Models*. In Proceedings of NeurIPS 2023.
3. Josh Achiam, Steven Adler, Sandhini Agarwal, et al. 2023. *GPT-4 Technical Report*. In Proceedings of NeurIPS 2023.

4. Supabase Inc. 2024. *pgvector: Open-Source Vector Similarity Search for Postgres*. Supabase Technical Documentation and Developer Reports 2024.

5. Google AI Platform Team. 2024. *Generative AI API Rate Limits and Quota Management*. Google Cloud Technical Documentation 2024.

6. Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020 (updated 2023–2024). *Retrieval-Augmented Generation for Knowledge Intensive NLP Tasks*. In Advances in Neural Information Processing Systems.

7. J. Johnson, M. Douze, and H. Jégou. 2017 (updated 2023). *Billion-Scale Similarity Search with GPUs*. IEEE Transactions on Big Data.

8. Render Engineering Team. 2024. *Managing Cold Starts in Serverless Deployments*. Render Technical Documentation 2024.

9. Supabase Inc. 2024. *Row Level Security and Service Role Key Architecture*. Supabase Security Documentation 2024.

10. Diyi Yang, Besmira Nushi, Ece Kamar, et al. 2023. *Human-Centered AI in Education: Principles and Practices*. ACM Conference on Fairness, Accountability, and Transparency (FAccT 2023).

11. Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. 2023. *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR 2023).

12. Tianyi Zhang, Varsha Kishore, Felix Wu, et al. 2023. *Knowledge-Augmented Language Models for Domain-Specific QA*. ACL 2023.

13. OpenAI. 2024. *GPT-4o: Multimodal Foundation Model Technical Overview*. OpenAI Technical Report 2024.

14. Gemini Team. 2024. *Gemini 1.5: Scaling Multimodal Reasoning with Long Context Windows*. Google DeepMind Technical Report 2024.

15. Li et al. 2023. *Multimodal AI Tutors: Integrating Vision and Language for STEM Education*. IEEE Transactions on Learning Technologies.

16. Amershi et al. 2023. *Guidelines for Human-AI Interaction*. CHI Conference Proceedings 2023.

17. Madaio et al. 2024. *Human-Centered AI Governance in Educational Institutions*. ACM Conference on AI, Ethics, and Society 2024.

18. Qin et al. 2023. *Tool Learning with Foundation Models*. NeurIPS 2023 Workshops.

19. Park et al. 2023. *Generative Agents: Interactive Simulacra of Human Behavior*. ACM UIST 2023.

20. Hong et al. 2024. *Multi-Agent LLM Systems: A Survey*. arXiv preprint 2024.

21. Dean et al. 2023. *Large-Scale Distributed Systems for AI Workloads*. Communications of the ACM.

22. Kleppmann. 2023. *Designing Data-Intensive Applications (Updated Cloud Patterns Edition)*. O'Reilly Media.

23. Bommasani et al. 2021 (updated 2023). *On the Opportunities and Risks of Foundation Models*. Stanford CRFM Report.

24. OpenAI. 2023. *GPT-4 System Card*. OpenAI Safety Report.

25. Zaharia et al. 2023. *Databricks Lakehouse Architecture for AI Workloads*. VLDB 2023.

26. Wang et al. 2023. *Self-Consistency Improves Chain-of-Thought Reasoning in Language Models*. ICLR 2023.

27. Li et al. 2024. *Reliable LLM Systems: Evaluation and Robustness Challenges*. arXiv preprint 2024.

28. Chen et al. 2023. *Evaluating Retrieval-Augmented Generation Systems: Metrics and Benchmarks*. EMNLP 2023.

29. Touvron et al. 2023. *LLaMA 2: Open Foundation and Fine-Tuned Chat Models*. Meta AI Technical Report.

30. Wei et al. 2022 (updated 2023). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2022.